

Introduction to Computer Architecture

Chapter 4 - 5

Pipeline Hazards

Hyungmin Cho

Department of Computer Science and Engineering
Sungkyunkwan University

Data Hazards in ALU Instructions

■ Data hazard example:

add ^{old} **x2**, x10, x11 ✕

...

sub ^{new} **x2**, x1, x3

and x12, **x2**, x5

or x13, x6, **x2**

add x14, x2, **x2**

sw x15, 100(**x2**)

hazard은 이전과 관련
여기서 obtain하기 때문

X2이 값이 바뀌는 것 WB,

Dependencies & Forwarding

이제나 add의 20에서 여전히 old value일, SW부터 new

sub ^{new}x2, x1, x3

and x12, ^{old}x2, x5

or x13, x6, ^{old}x2

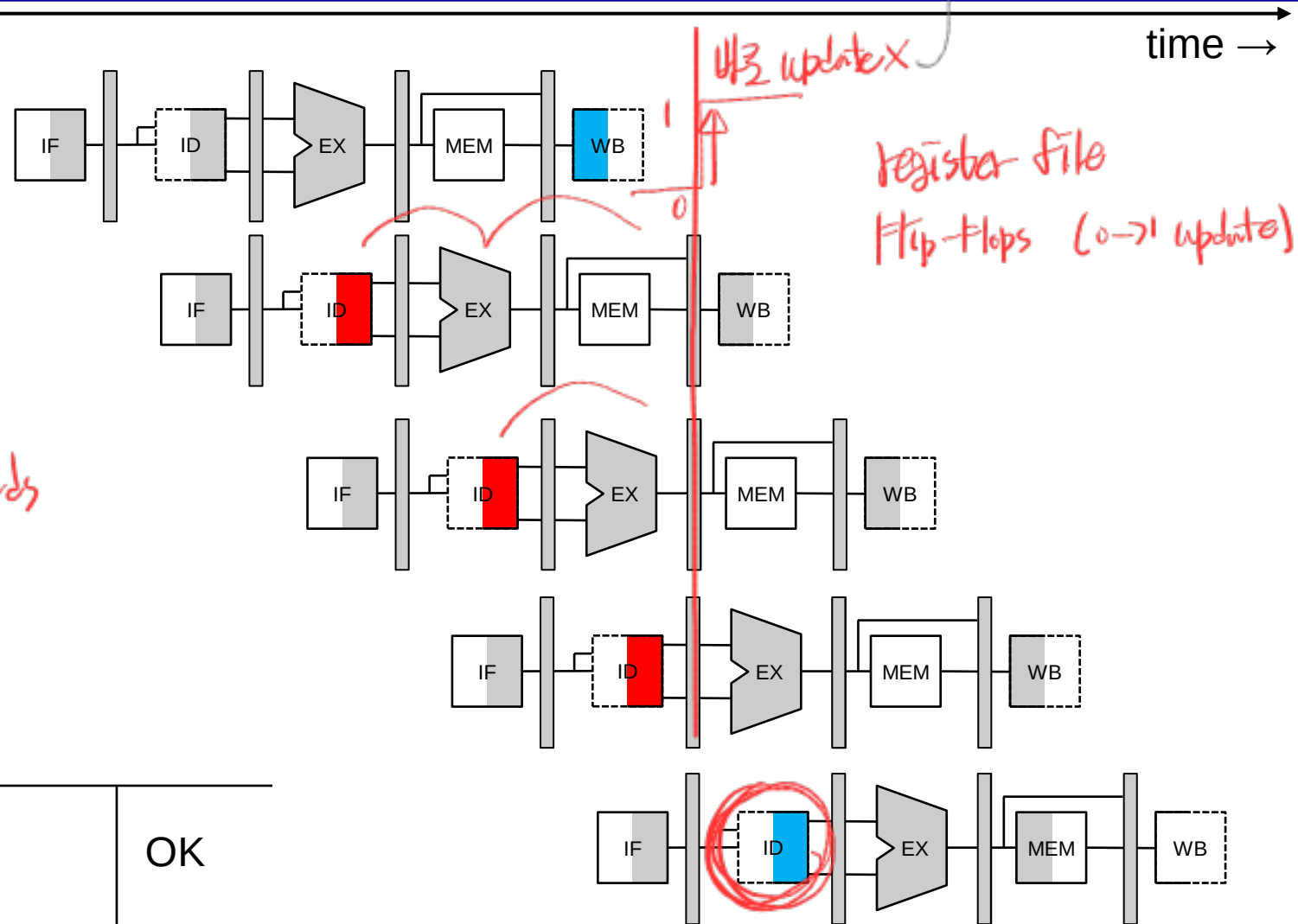
add x14, x2, ^{old}x2

sw x15, 100, ^{new}x2

hazards

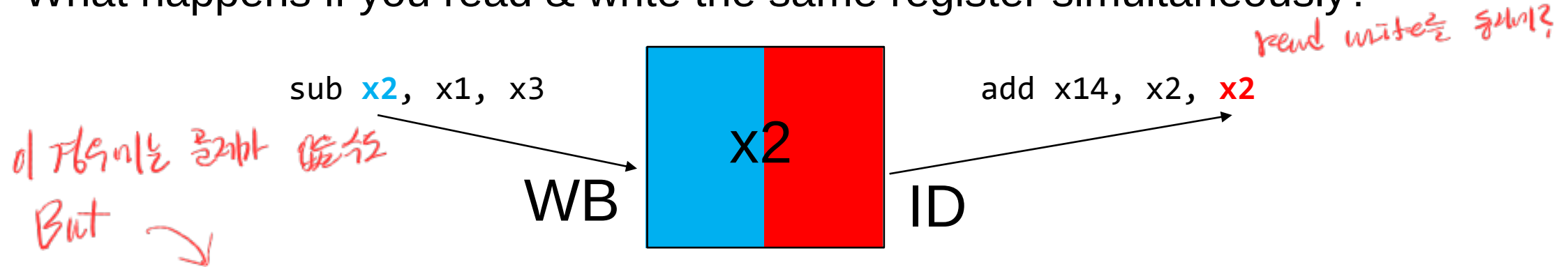
OK

SW부터 new 업데이트
x2 사용가능 (4cycle)



Same Register Read & Write

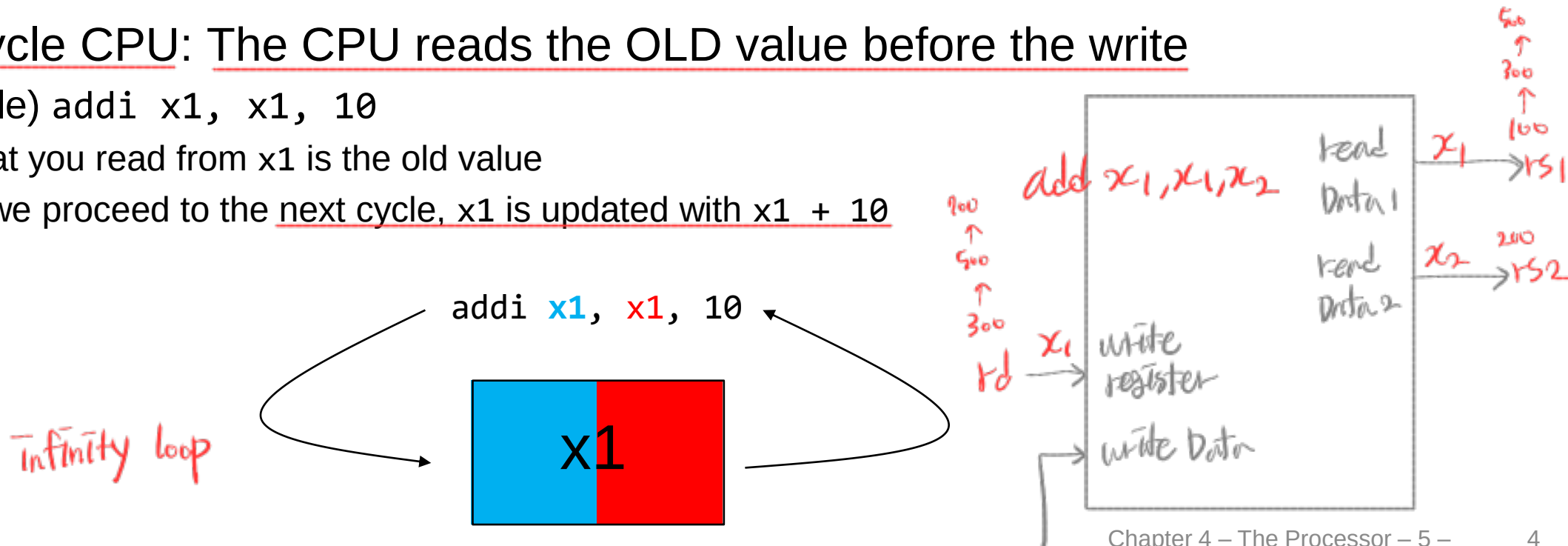
- What happens if you read & write the same register simultaneously?



- Single-cycle CPU: The CPU reads the OLD value before the write

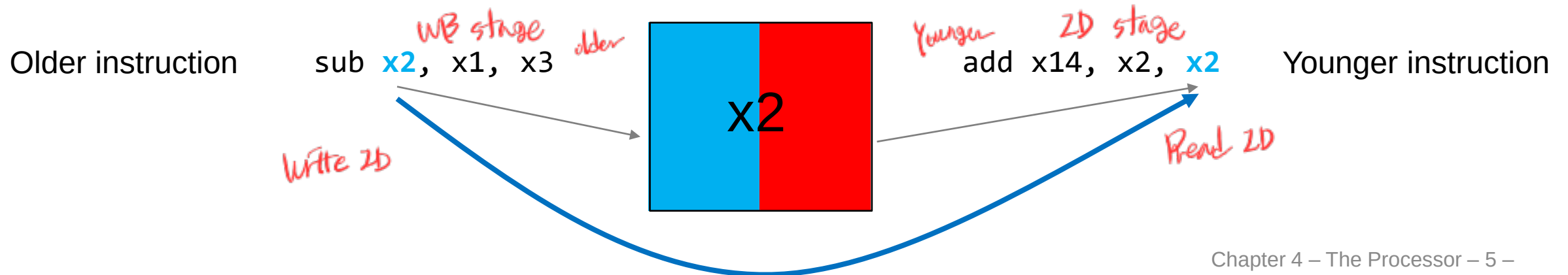
❖ Example) addi x1, x1, 10

- What you read from x1 is the old value
- As we proceed to the next cycle, x1 is updated with $x1 + 10$

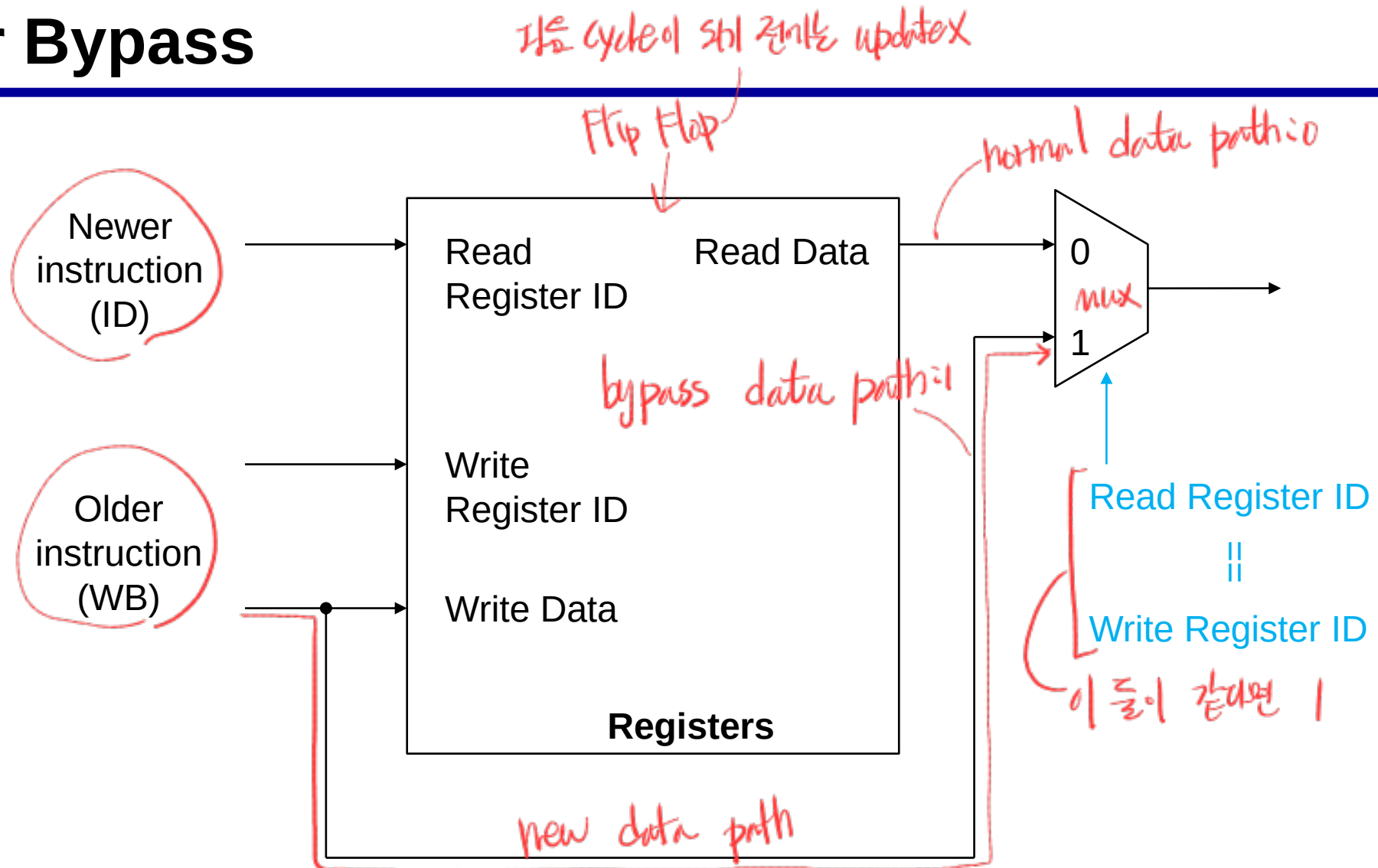


Same Register Read & Write

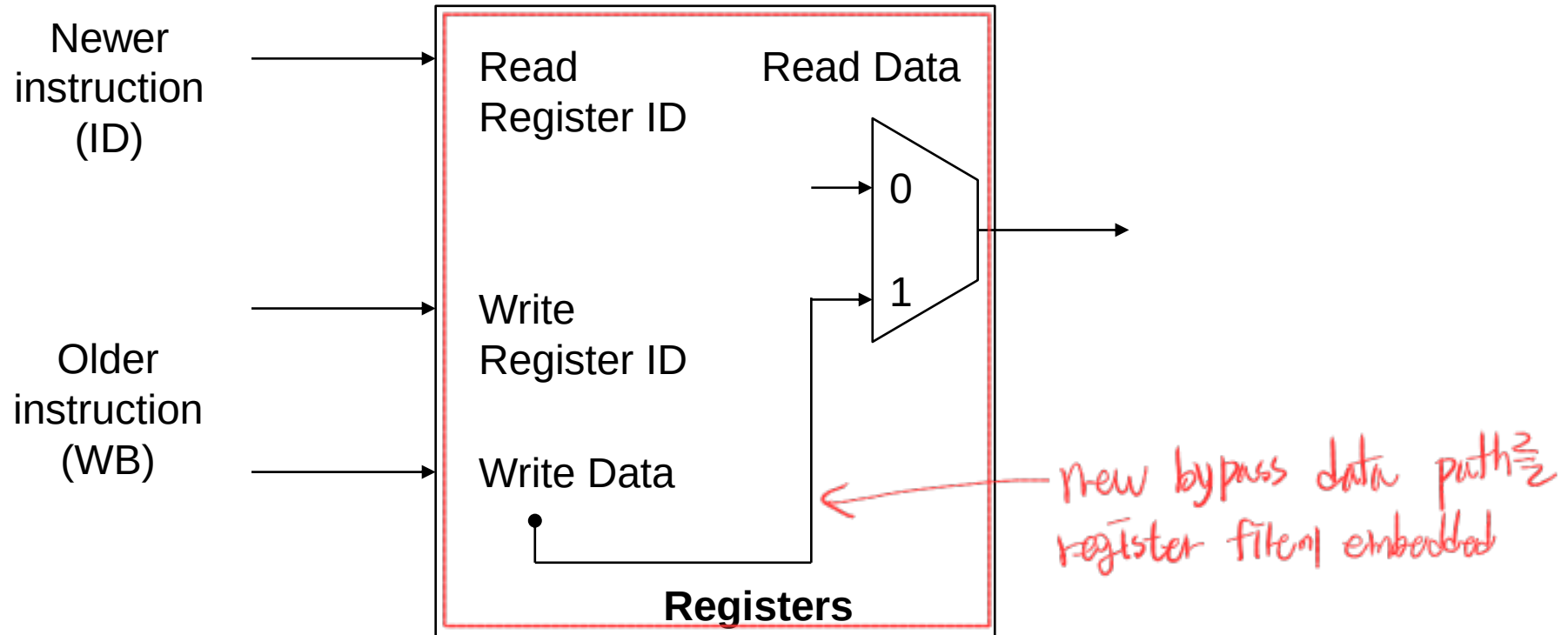
- Pipelined CPU: Let the CPU to read the NEW value written in the current cycle
 - ❖ An instruction does not read and write its source/destination registers at the same time
 - Read: ID stage, Write: WB stage
 - ❖ From the register file's perspective,
 - Read: Younger instruction
 - Write: Older instruction
 - ❖ → Bypass the older instruction's value to the younger instruction



Register Bypass



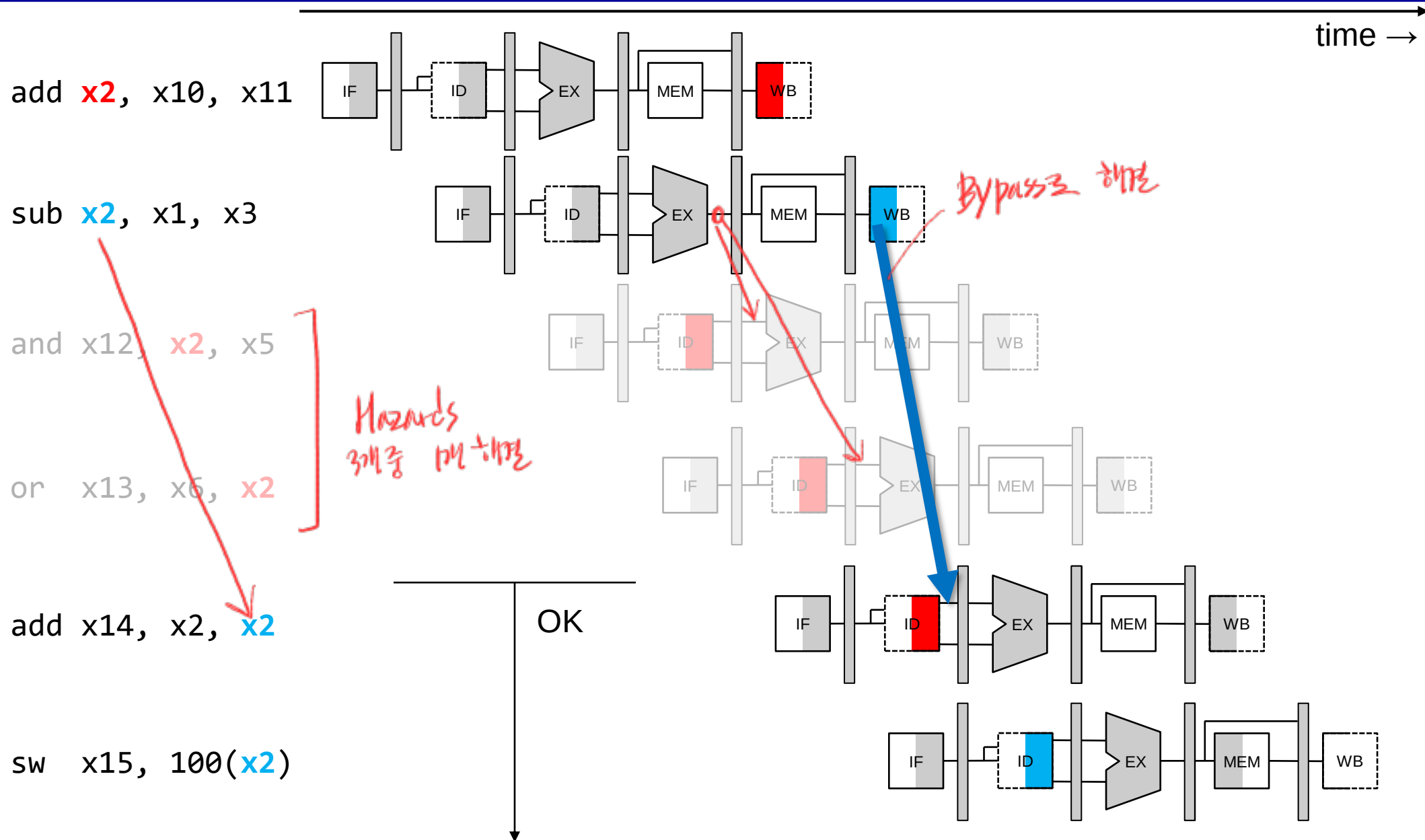
Register Bypass



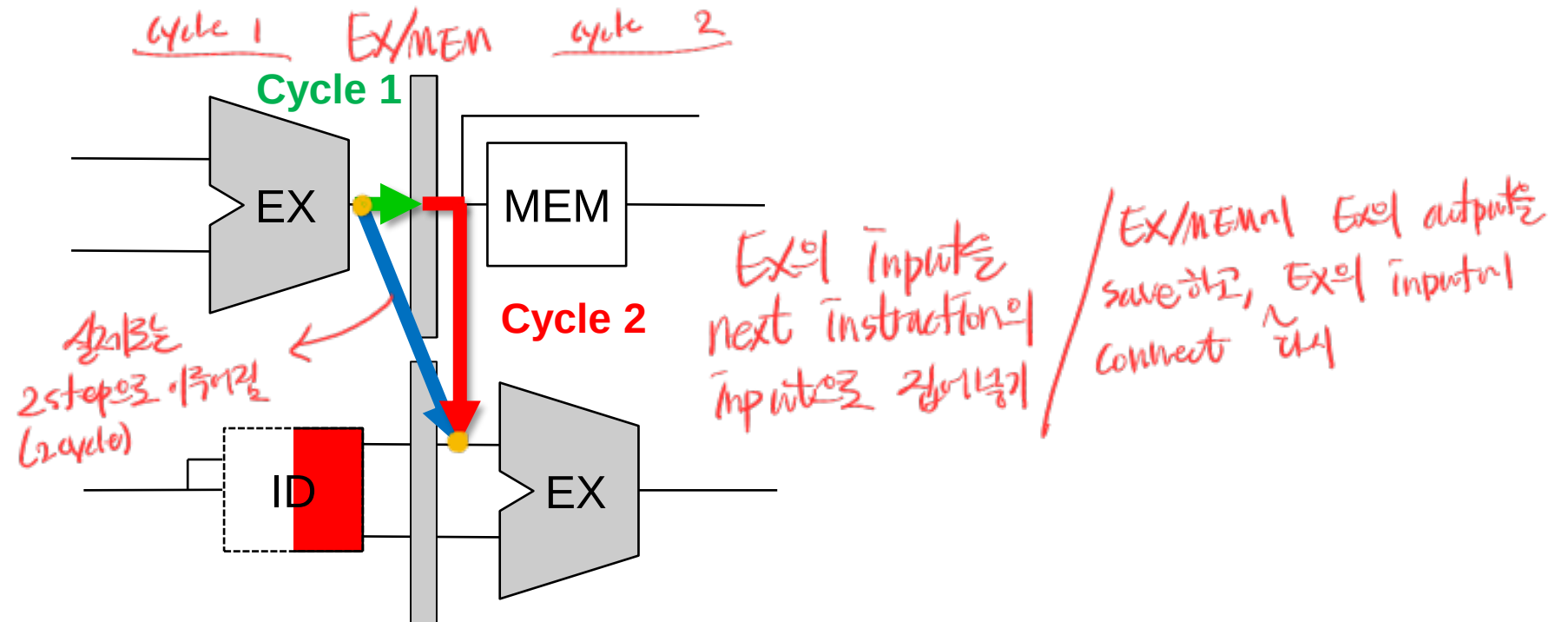
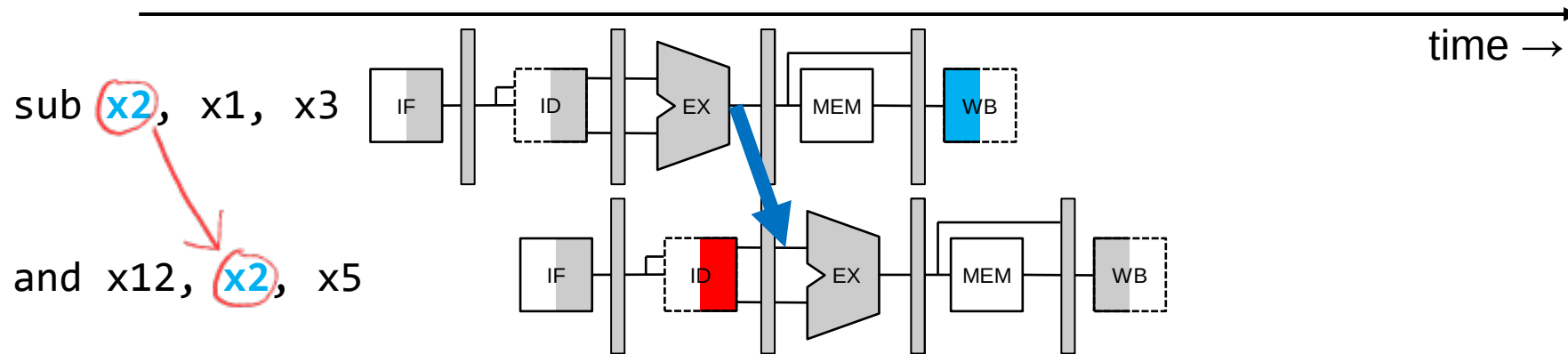
- From now on, we assume the bypass circuit is embedded in the register file

register unit
bypass circuit

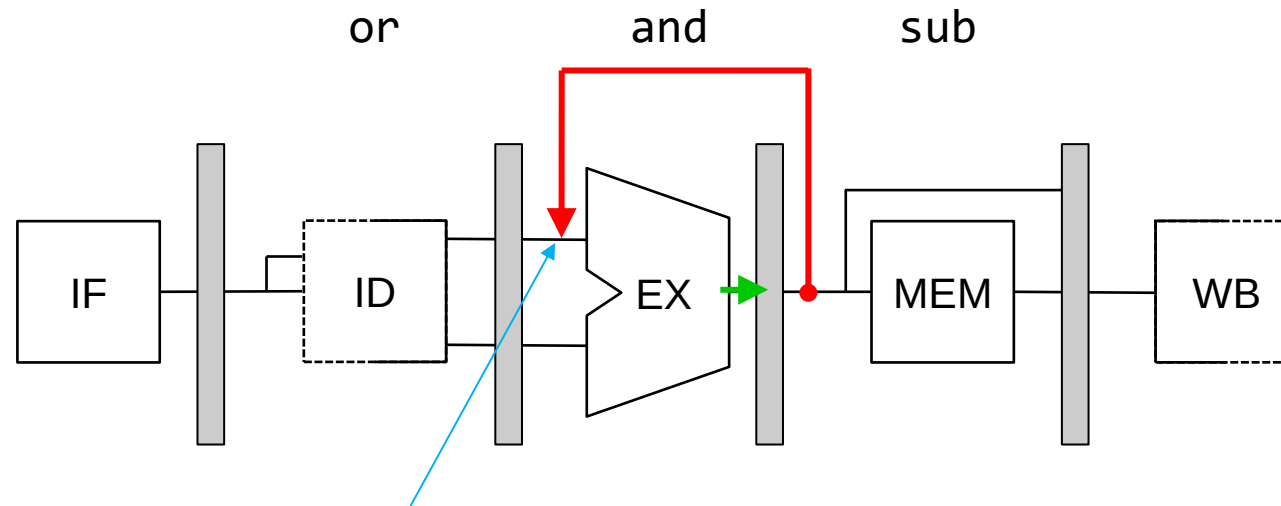
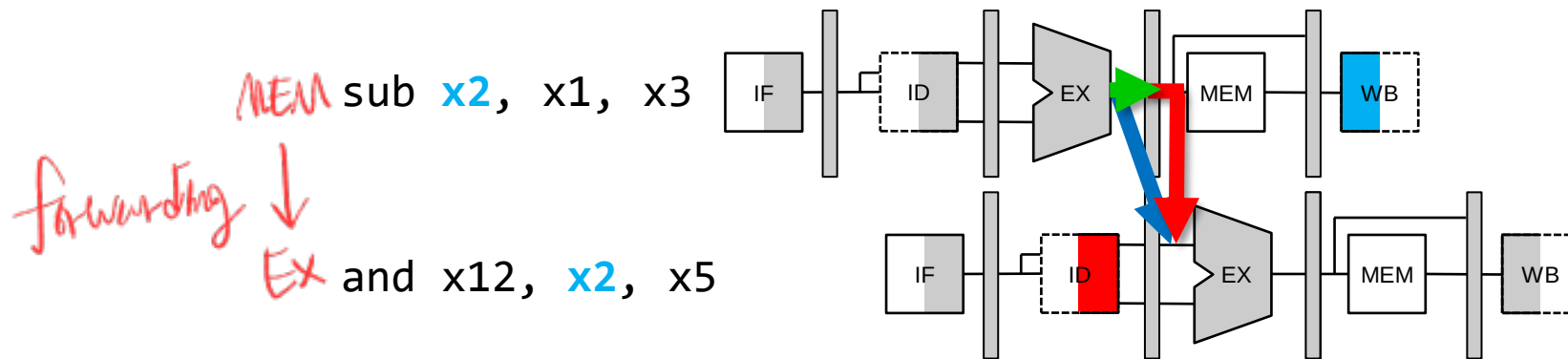
Dependency Resolving using Register Bypass



Register Forwarding



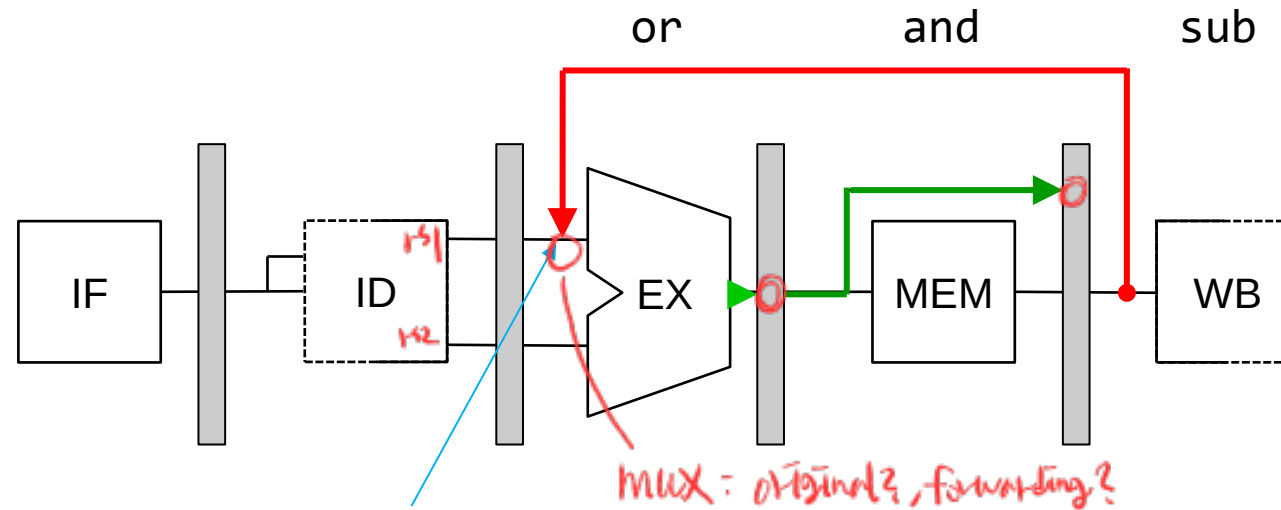
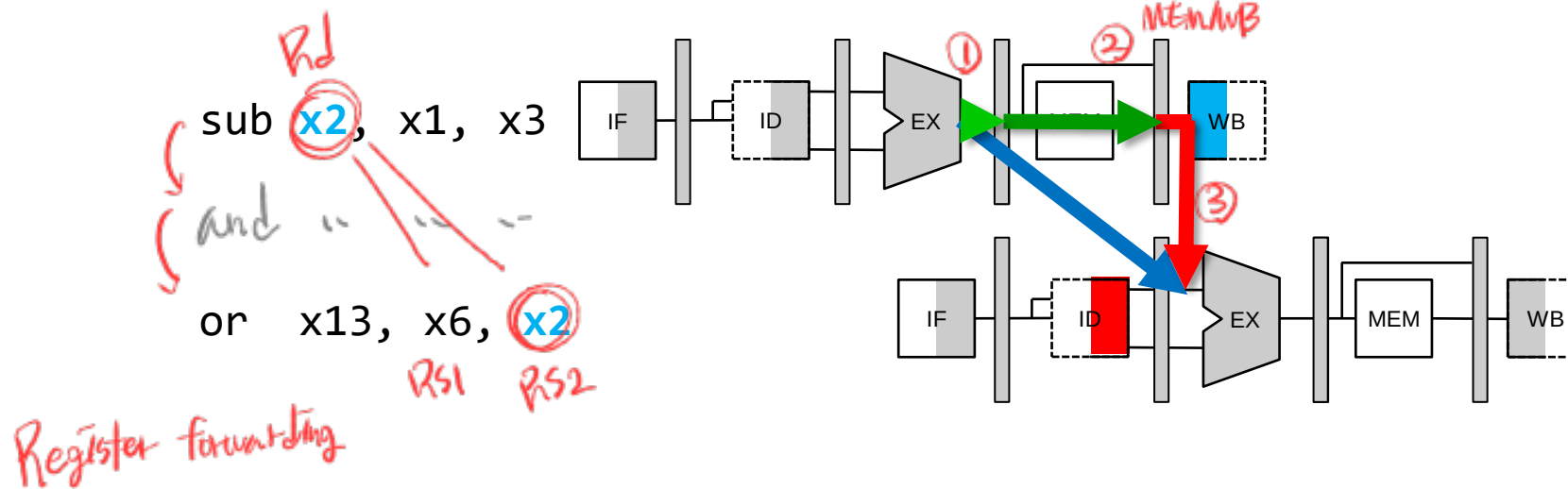
Register Forwarding



Need to decide when to use forwarding

Register Forwarding

3 cycle

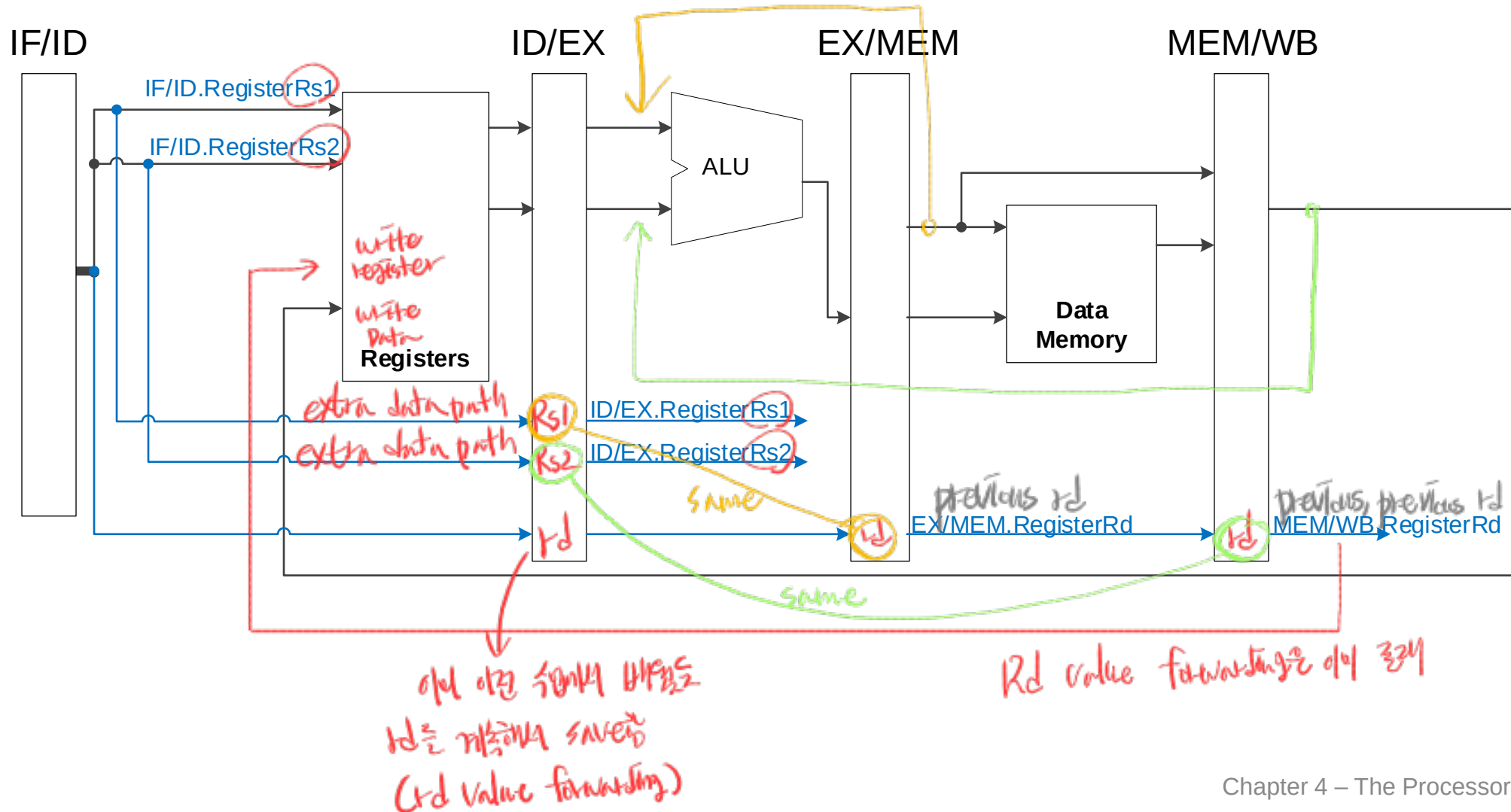


output = EX/memal save,
memal save, EXel
Inputal connect

Need to decide when to use forwarding

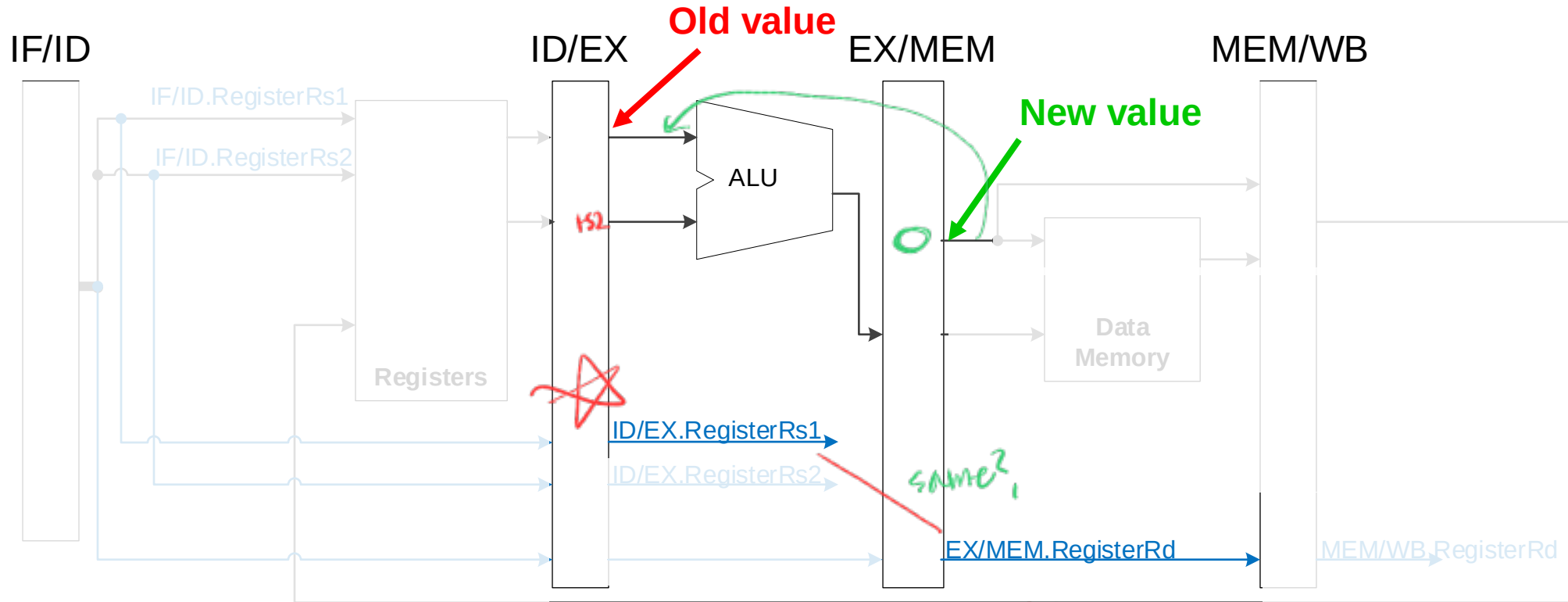
Detecting the Need for Forwarding

- Pass register numbers along the pipeline



Detecting the Need for Forwarding

- e.g., ID/EX.RegisterRs1 = register number for rs1 in ID/EX



Register Forwarding Condition

■ Data hazards when

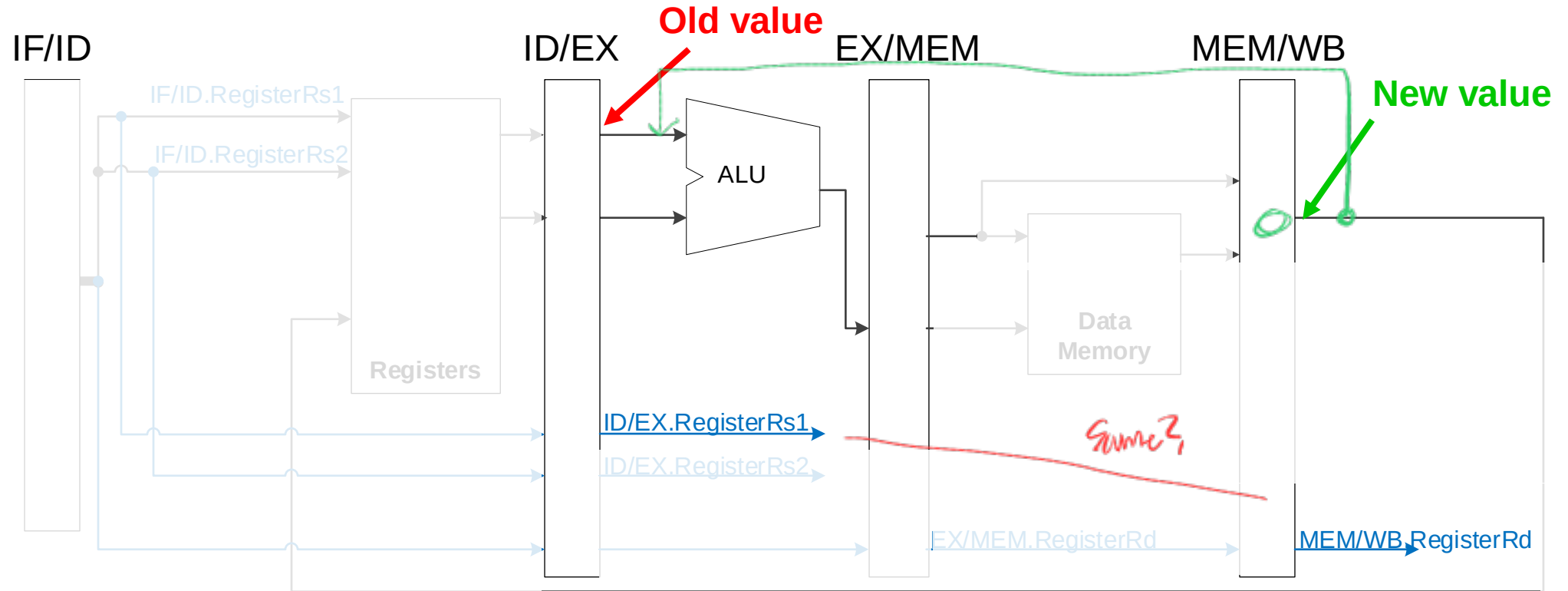
❖ 1a. $\text{EX/MEM.RegisterRd} = \text{ID/EX.RegisterRs1}$

❖ 1b. $\text{EX/MEM.RegisterRd} = \text{ID/EX.RegisterRs2}$

→ “EX Hazard”

Forward from
EX/MEM
pipeline reg

Detecting the Need for Forwarding



Register Forwarding Condition

■ Data hazards when

❖ 1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1

❖ 1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2

→ "EX Hazard"

IF ID T EX T MEM WB
Rs1 = Rd
Rs2 = Rd

Forward from
EX/MEM
pipeline reg

MEM → EX 82

❖ 2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1

❖ 2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2

→ "MEM Hazard"

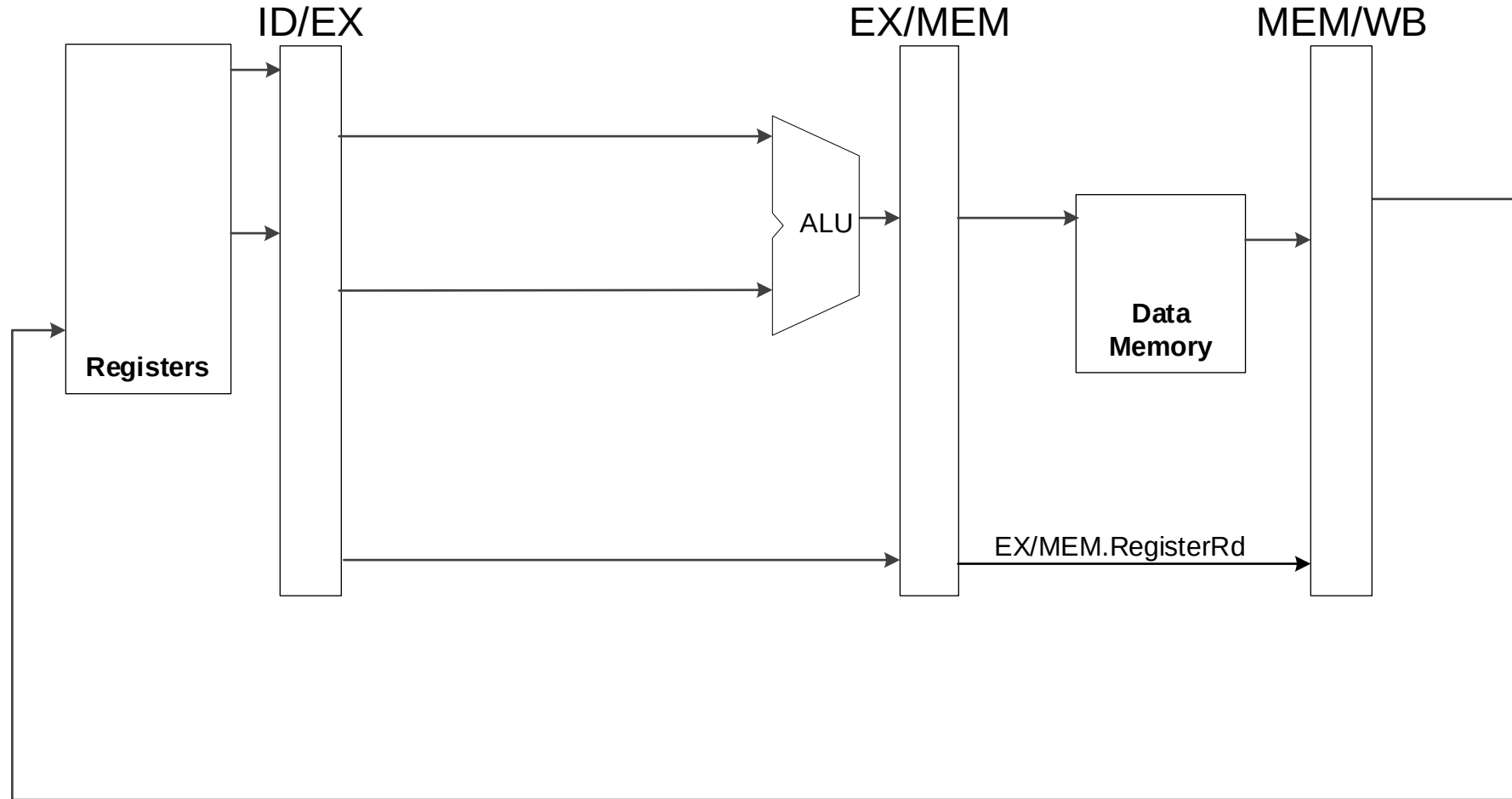
IF ID T EX T MEM T WB
Rs1 = Rd
Rs2 = Rd

Forward from
MEM/WB
pipeline reg

WB → EX 82

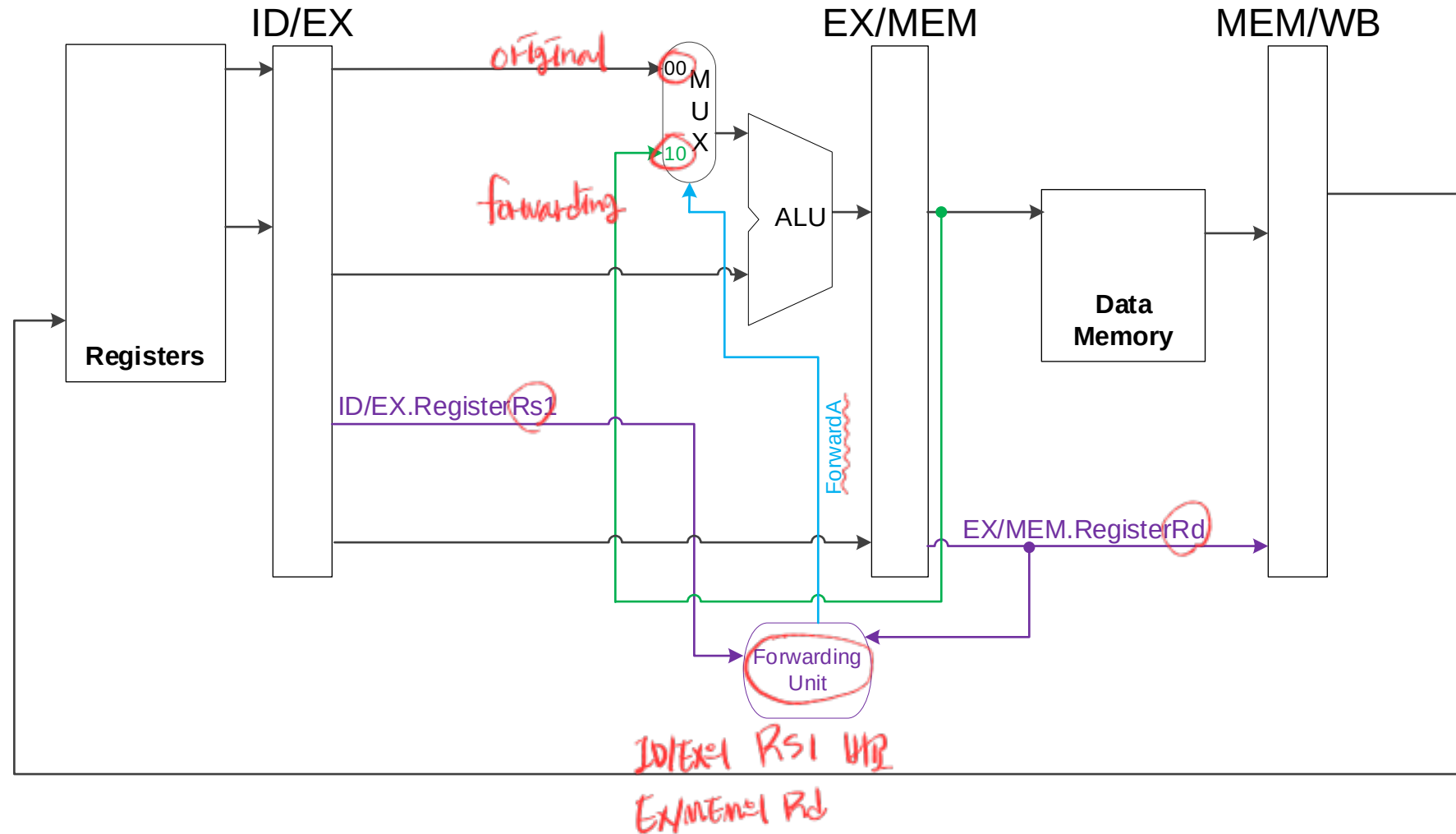
교의를 따름

Forwarding Paths – Original Datapath

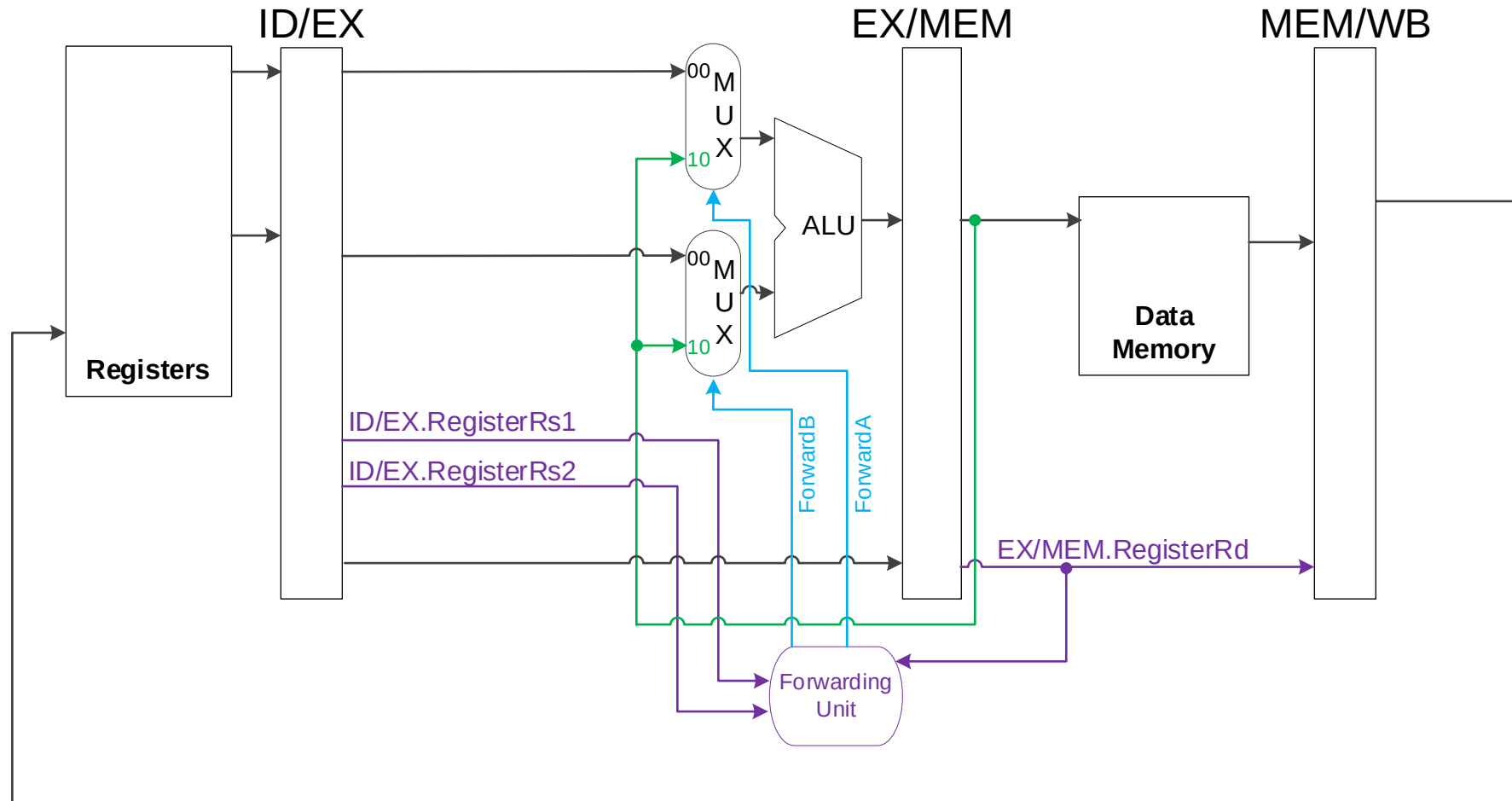


Forwarding Paths – Original Datapath

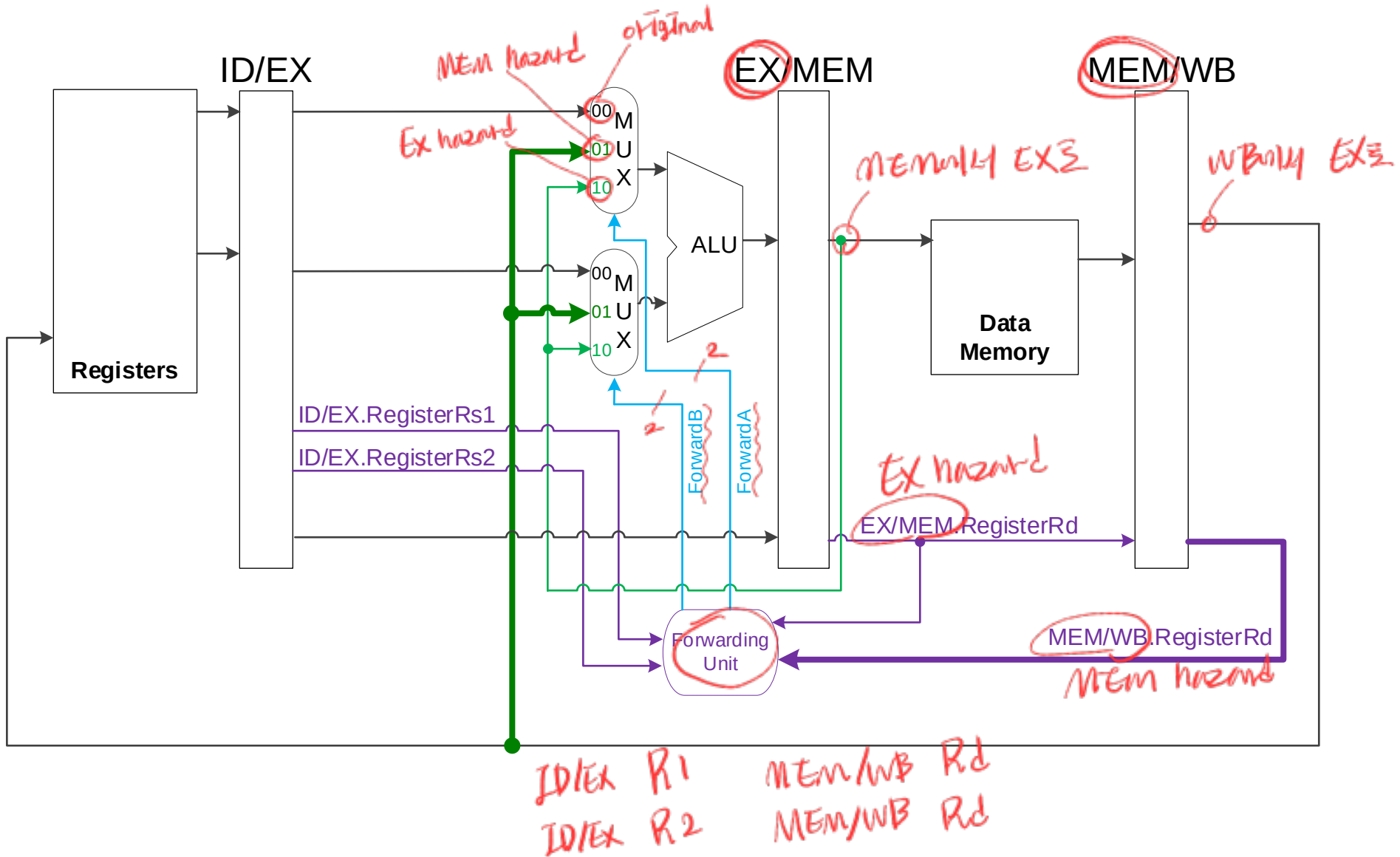
EX Hazard (CS)



Forwarding Paths – EX Hazard Only *RS 22 3/1*



Forwarding Paths – EX & MEM Hazards



Forwarding Conditions – Rs1

- Default (No forwarding)
 - ❖ ForwardA = 00
- EX hazard
 - ❖ if (EX/MEM.RegisterRd = ID/EX.RegisterRs1)
ForwardA = 10
- MEM hazard
 - ❖ if (MEM/WB.RegisterRd = ID/EX.RegisterRs1)
ForwardA = 01

Detecting the Need to Forward

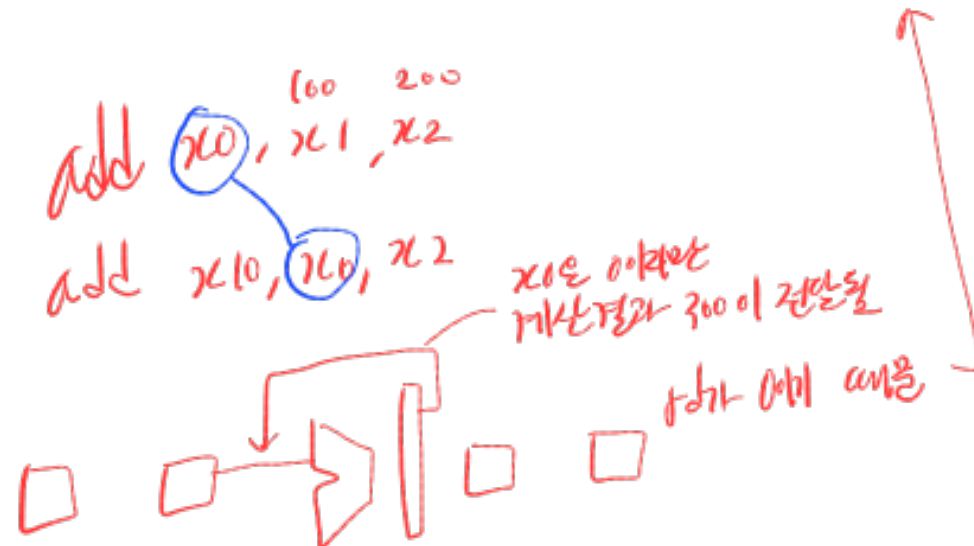
- But only if there is an actual register write !

그냥 forward만 하려하면 Instruction이
이런 type 인지를 알 수 없게
sw나 swl은 Register가 메모로 옮겨져

❖ EX/MEM.RegWrite, MEM/WB.RegWrite
Rd = 1 or = 1

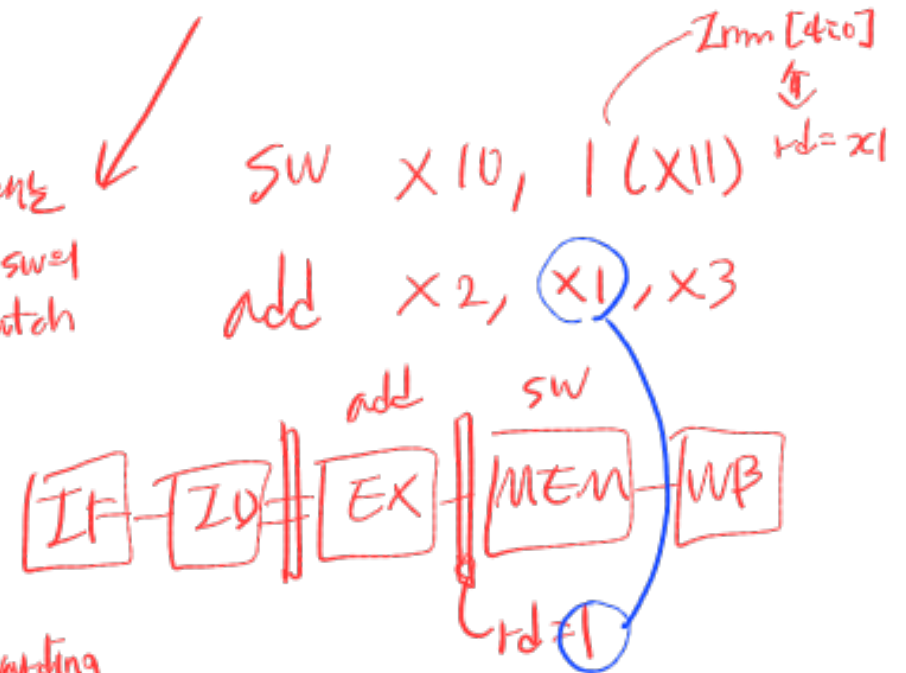
- ... and only if **Rd is not zero** !

❖ EX/MEM.RegisterRd $\neq 0$,
MEM/WB.RegisterRd $\neq 0$



*But 같은
add instruction sw의
rd field가 match*

*이런 forwarding
conditional이 따르면
forwarding이 이루어질
것이지만 Register를
확실히*



add x0, x1, x2
sub x10, x0, x3

*Incorrect execution
x0은 계속 이전 값을*

*x10
(x1+x2) - x3*

Forwarding Conditions – Rs1

- Default (No forwarding)

- ❖ ForwardA = 00

- EX hazard

- ❖ if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))

- ForwardA = 10

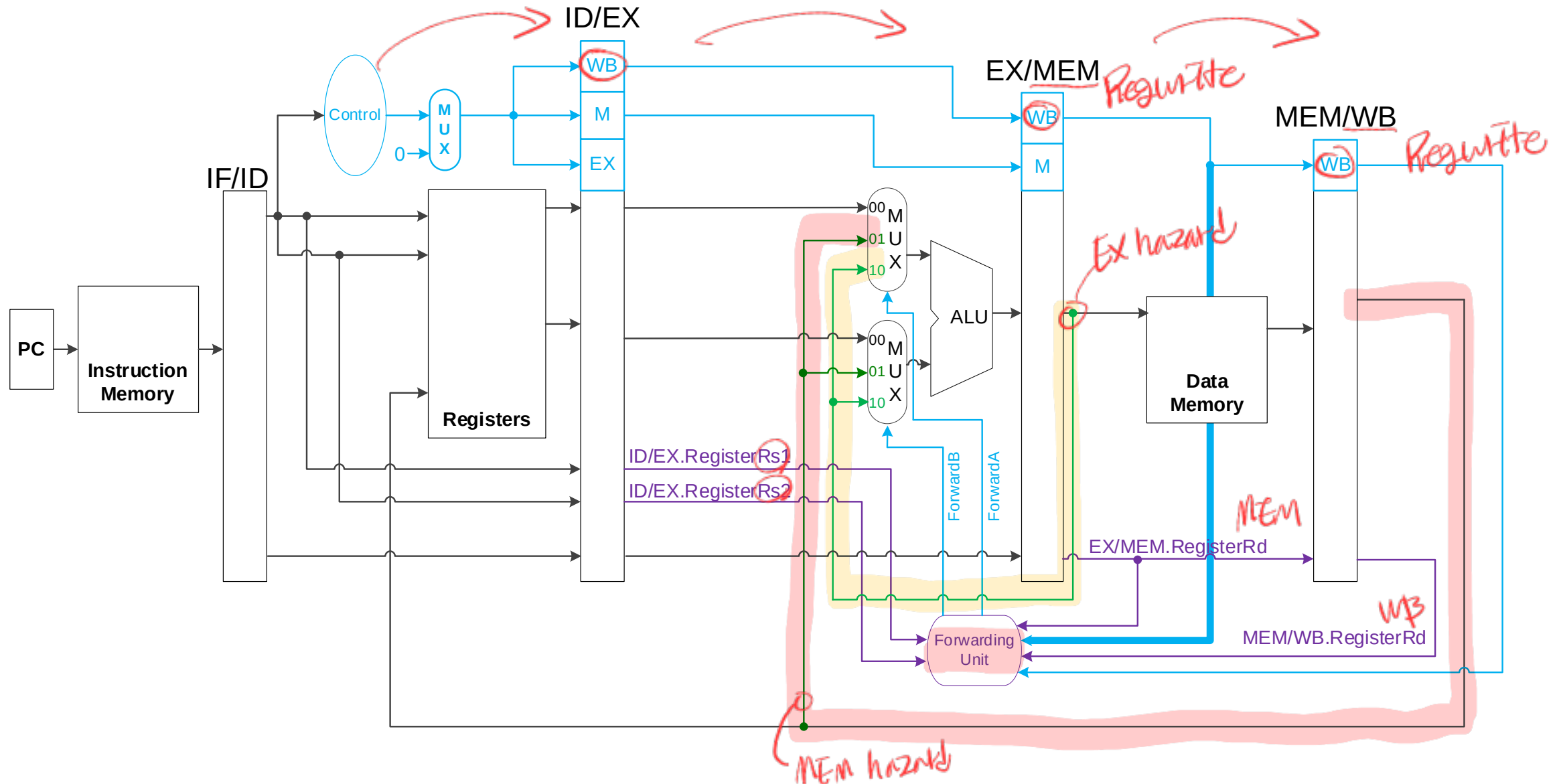
- MEM hazard

- ❖ if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and (MEM/WB.RegisterRd = ID/EX.RegisterRs1))

- ForwardA = 01



Datapath with Forwarding



Double Data Hazard

- Consider the following sequences:

add x1, x1, x2
add x2, x2, x3
add x1, x1, x4

MEM Hazard

add x1, x1, x2
add x1, x2, x3
add x1, x1, x4

MEM Hazard
EX Hazard

- Both hazards occur

- ❖ Want to use the most recent one

EX hazard is priority ↑

- Revise MEM hazard condition

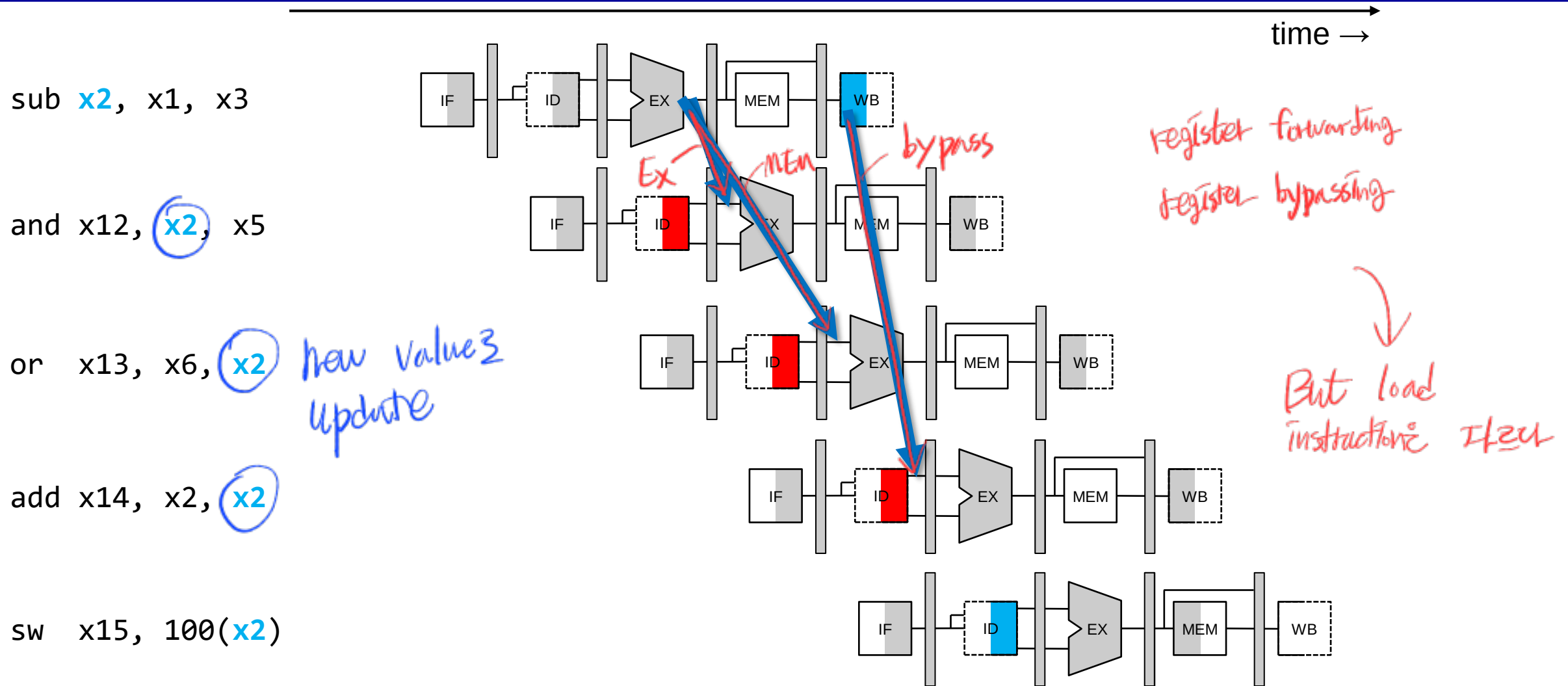
- ❖ Forward MEM hazard only when there is no EX hazard.

Revised Forwarding Condition

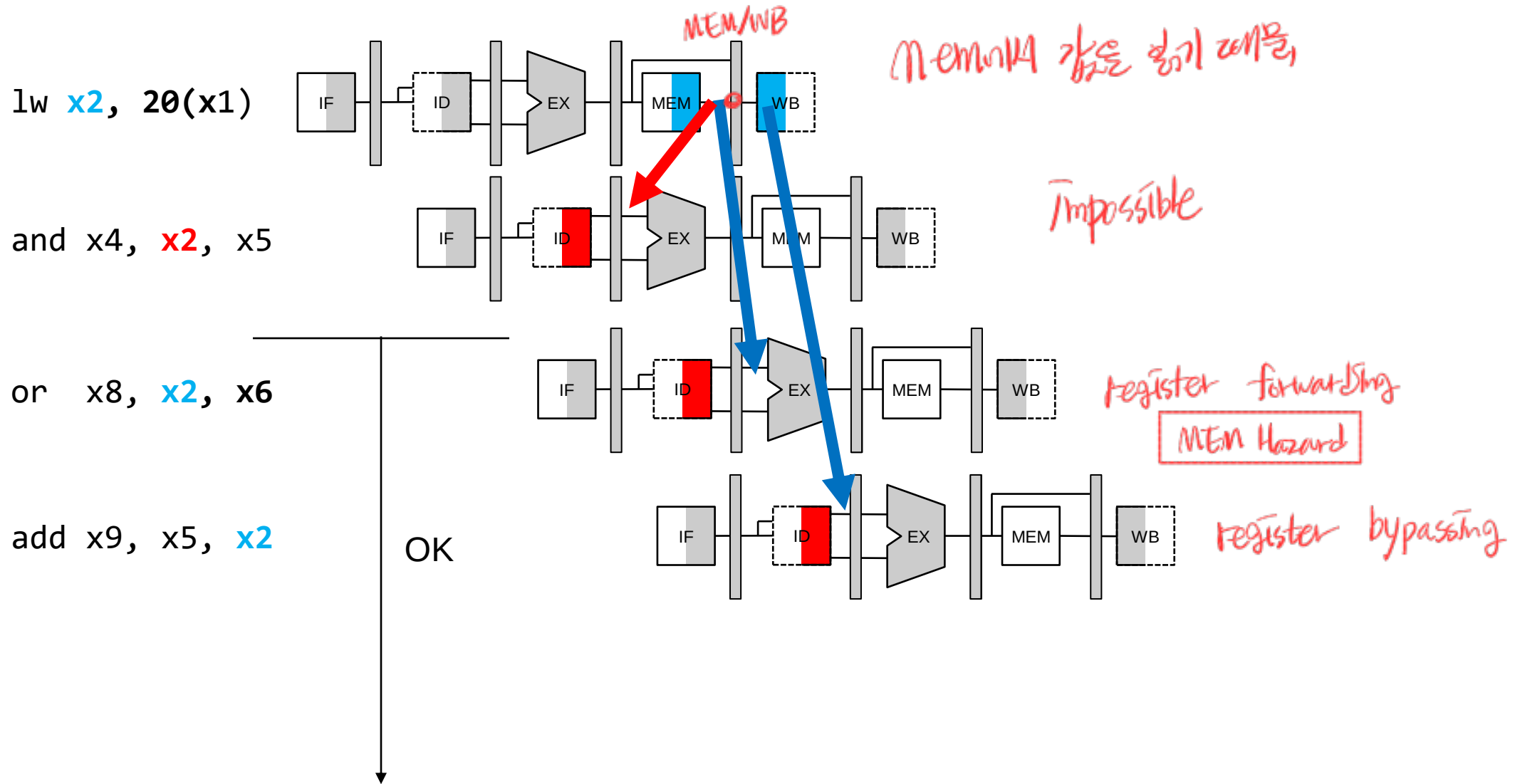
- EX hazard *priority*
 - ❖ if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
ForwardA = 10
- MEM hazard
 - ❖ if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and no EX hazard
and (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
ForwardA = 01

Same for Rs2...

Data Hazard Between R-Type Arithmetic Instructions



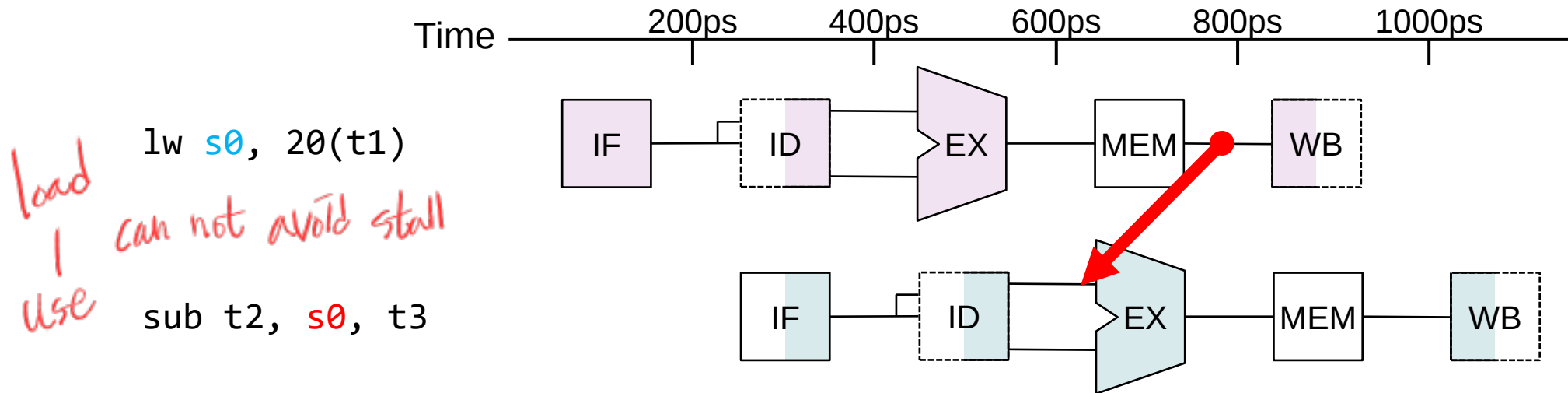
Load-Use Data Hazard



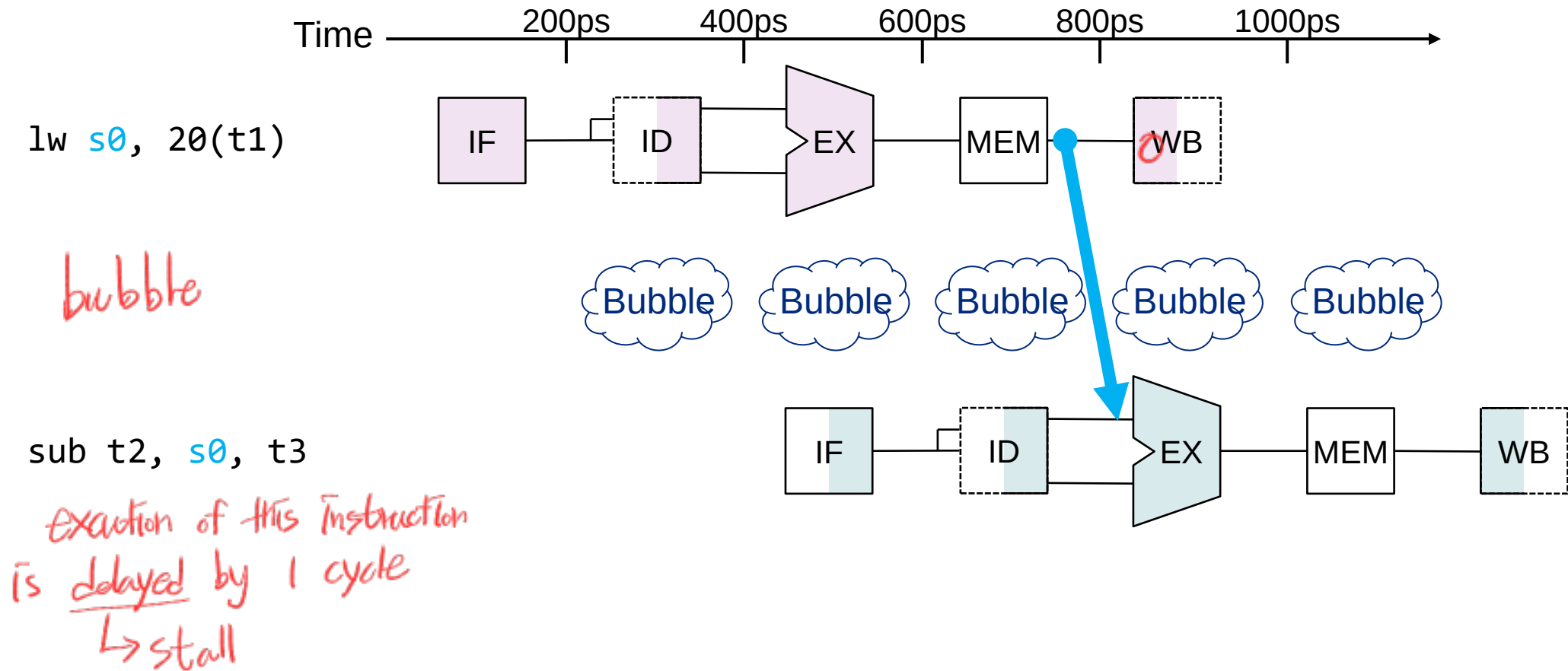
Load-Use Data Hazard

- Can't always avoid stalls by forwarding

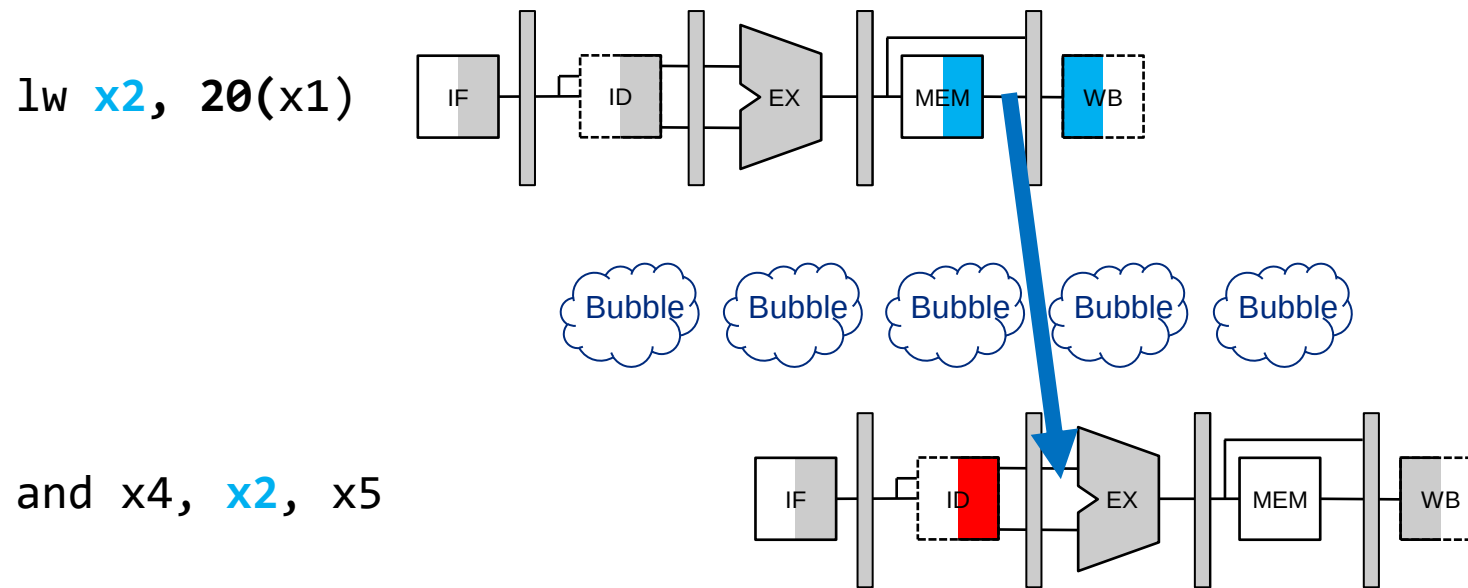
- ❖ If value not computed when needed
- ❖ Can't forward backward in time!



Load-Use Data Hazard



Load-Use Data Hazard

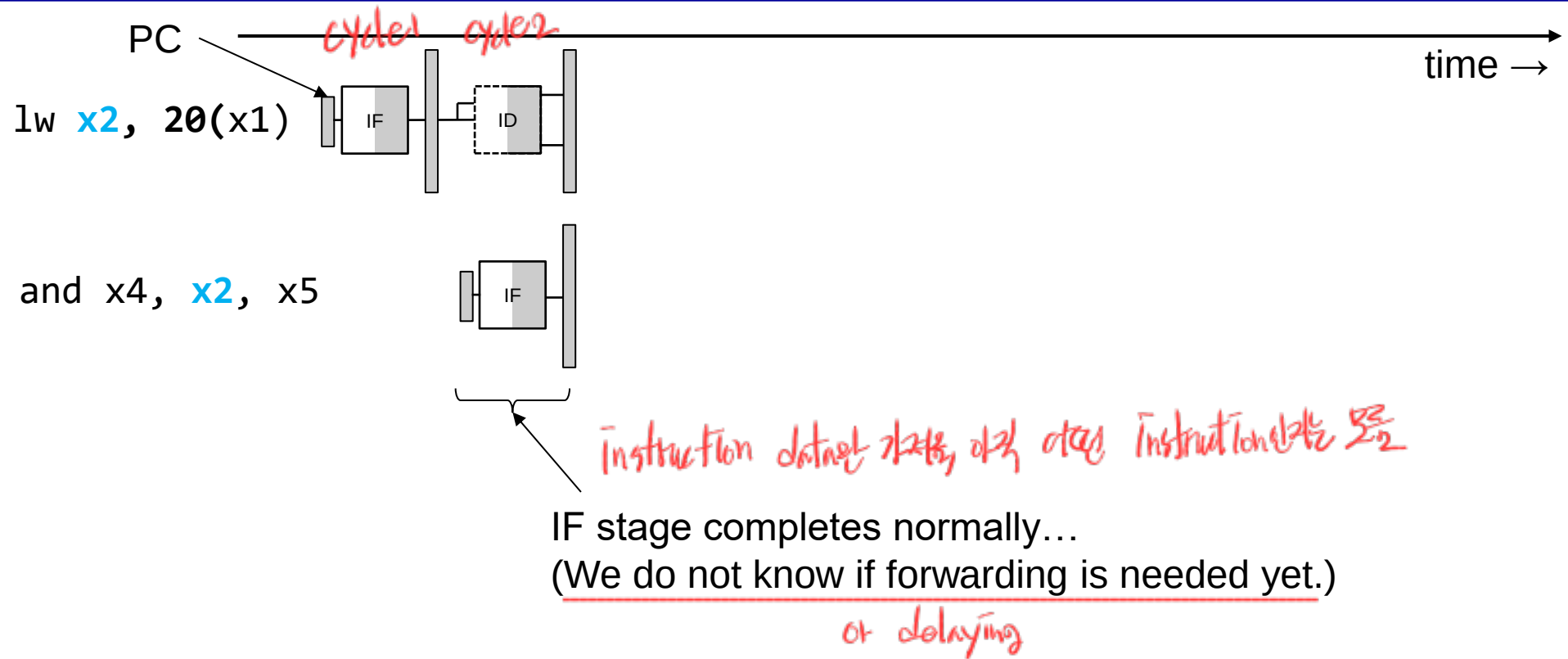


Need to stall 1 cycle

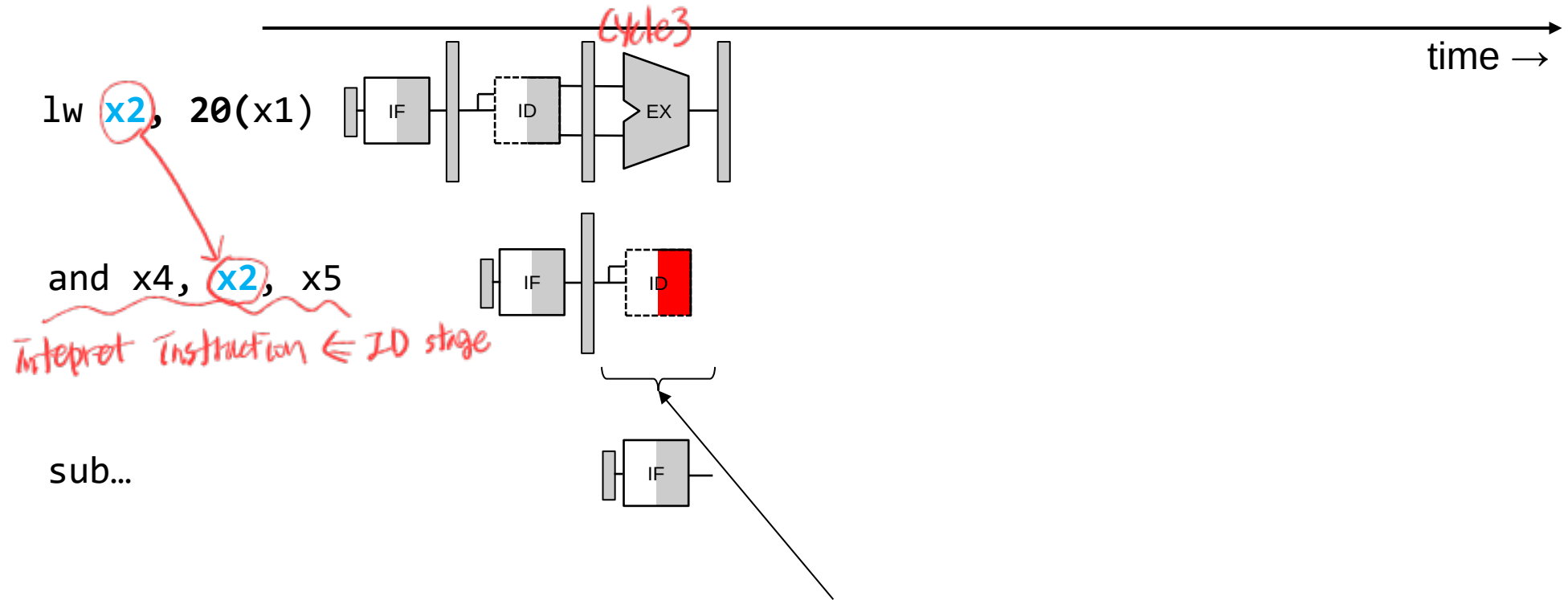
Let's see how it exactly works...

bubble은 dynamically
Insert될, 언제 필요한지 모르기
때문

Stall for Load-Use Data Hazard (1)



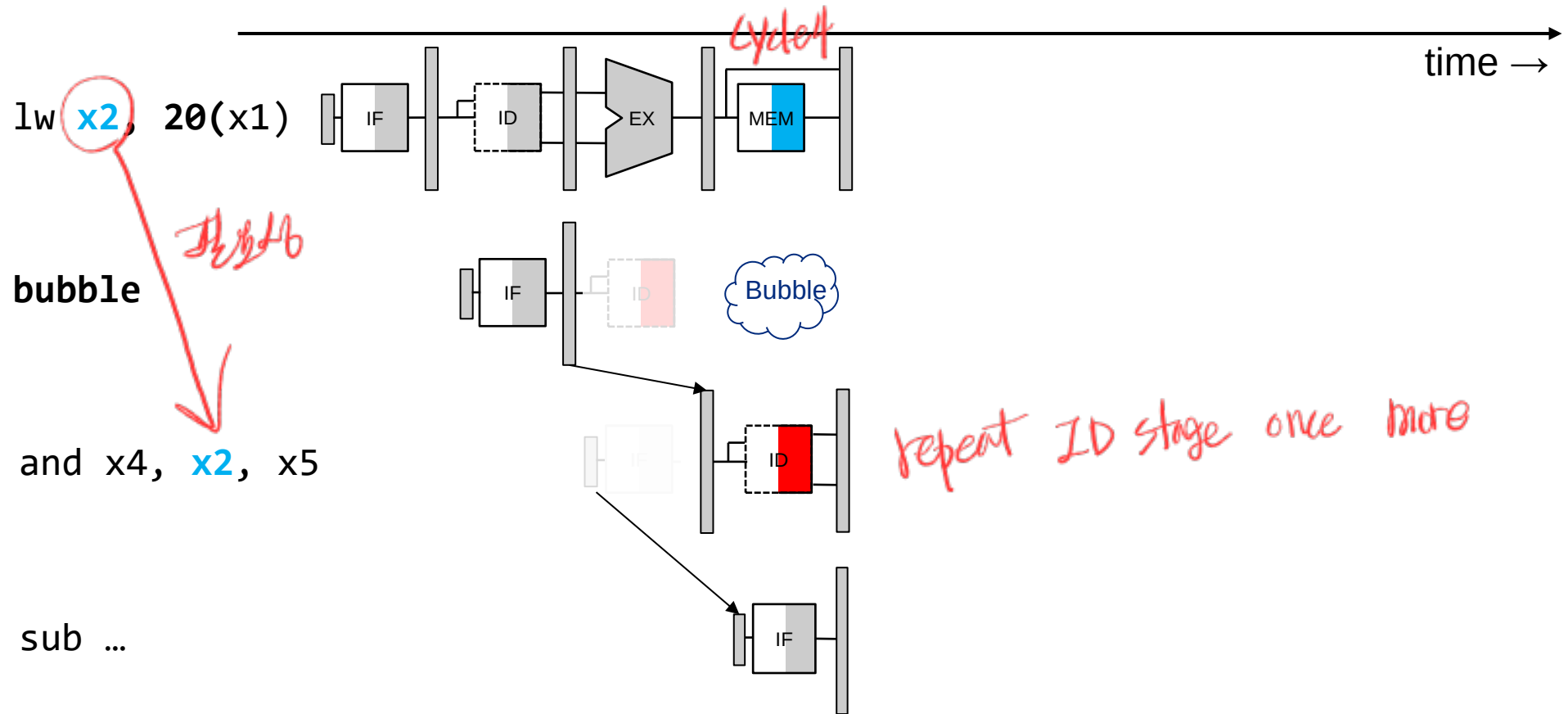
Stall for Load-Use Data Hazard (2)



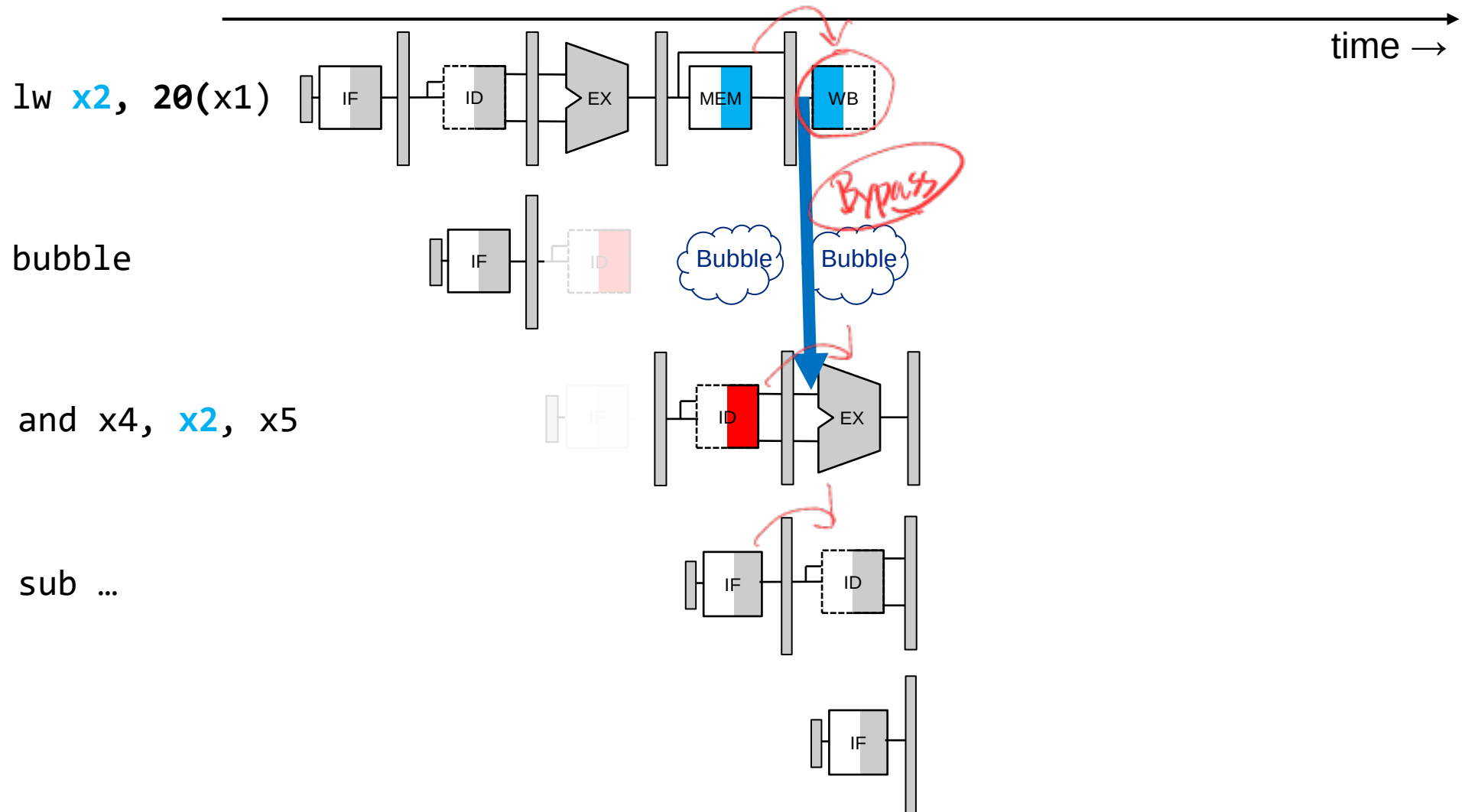
At the ID stage, we can identify if forwarding is required.

This instruction need to stay at the ID stage for 1 more cycle

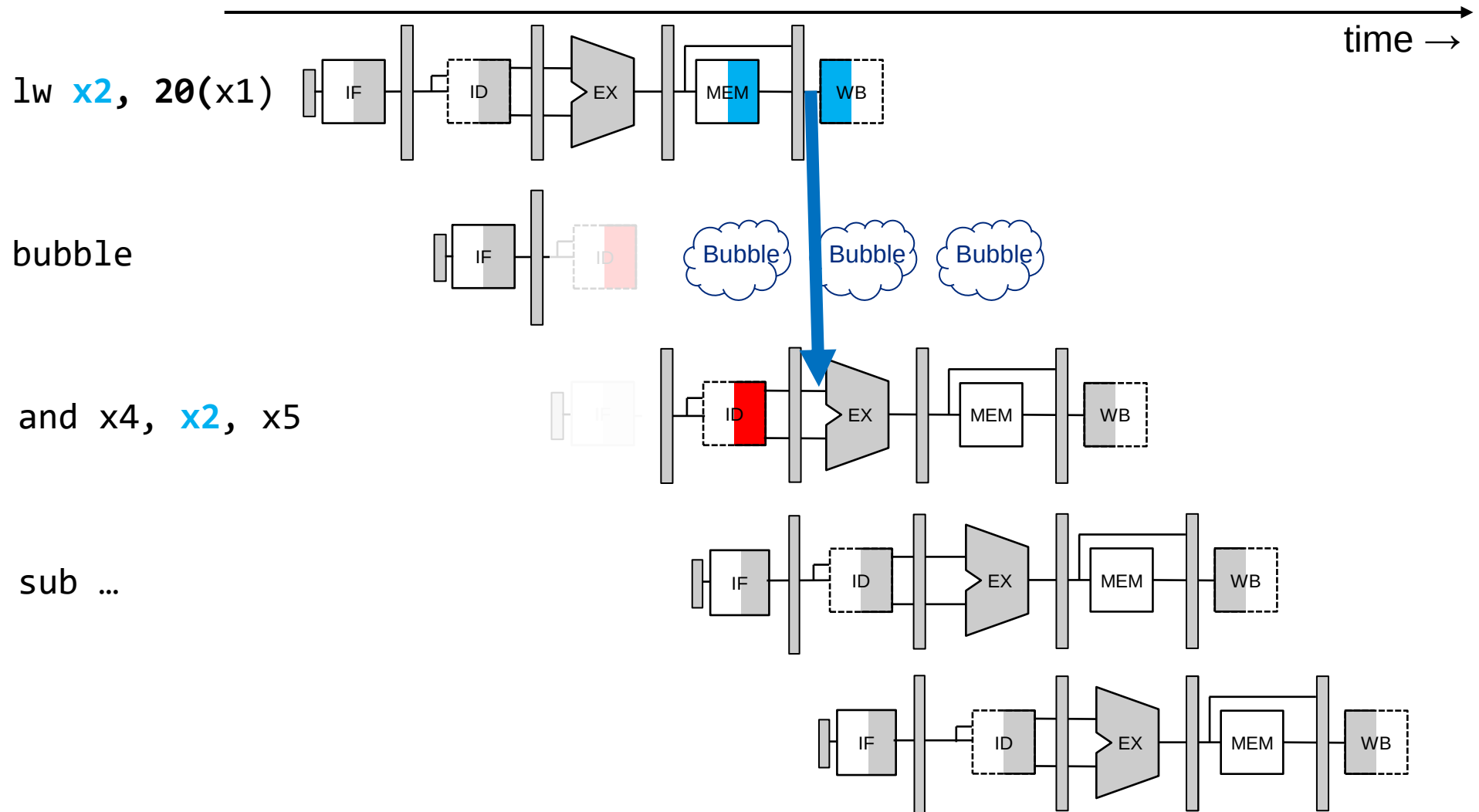
Stall for Load-Use Data Hazard (3)



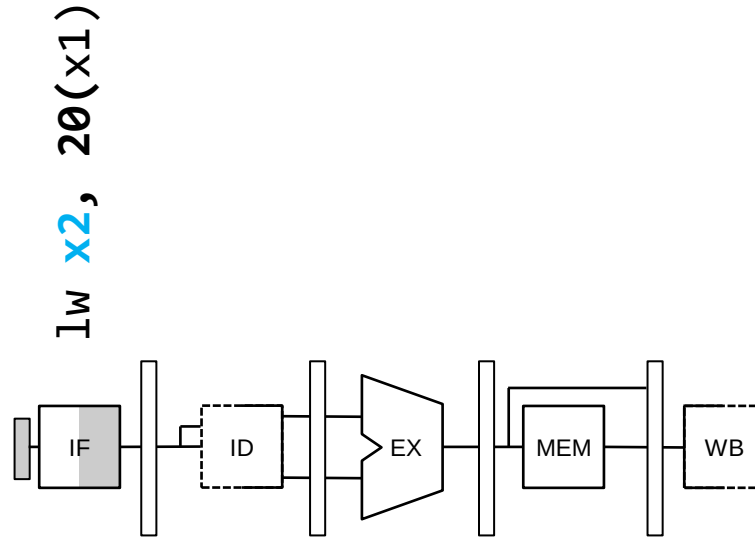
Stall for Load-Use Data Hazard (4)



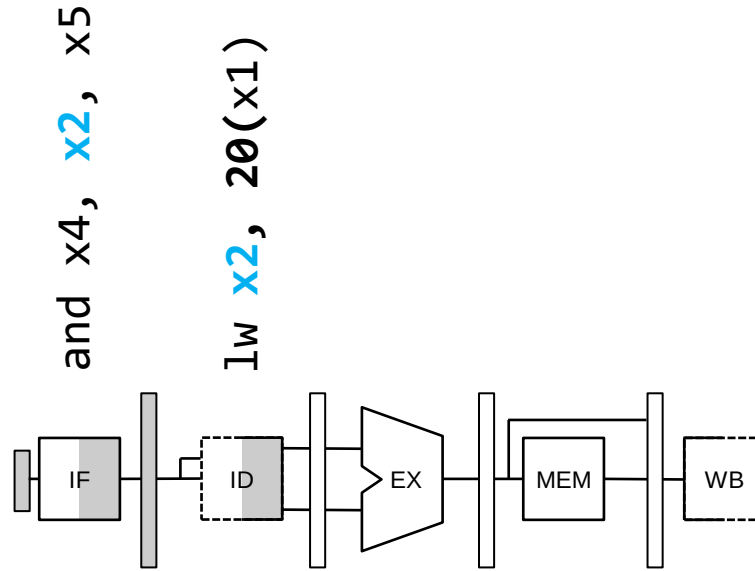
Stall for Load-Use Data Hazard (5)



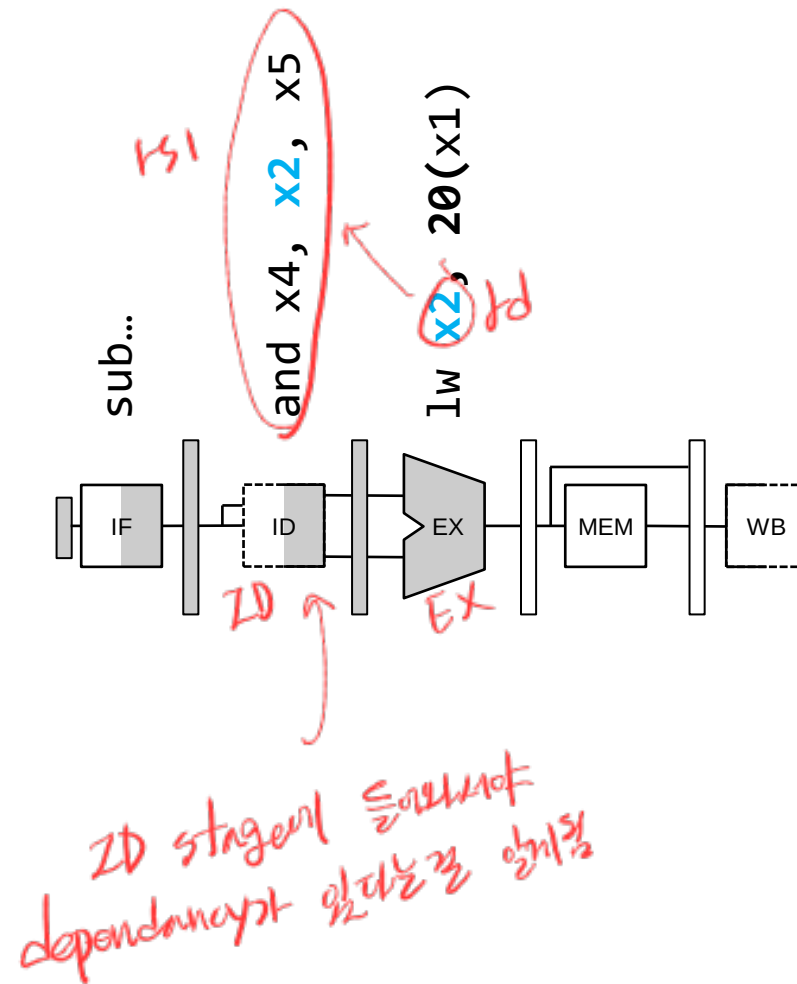
Stall for Load-Use Data Hazard – Different View: Cycle 1



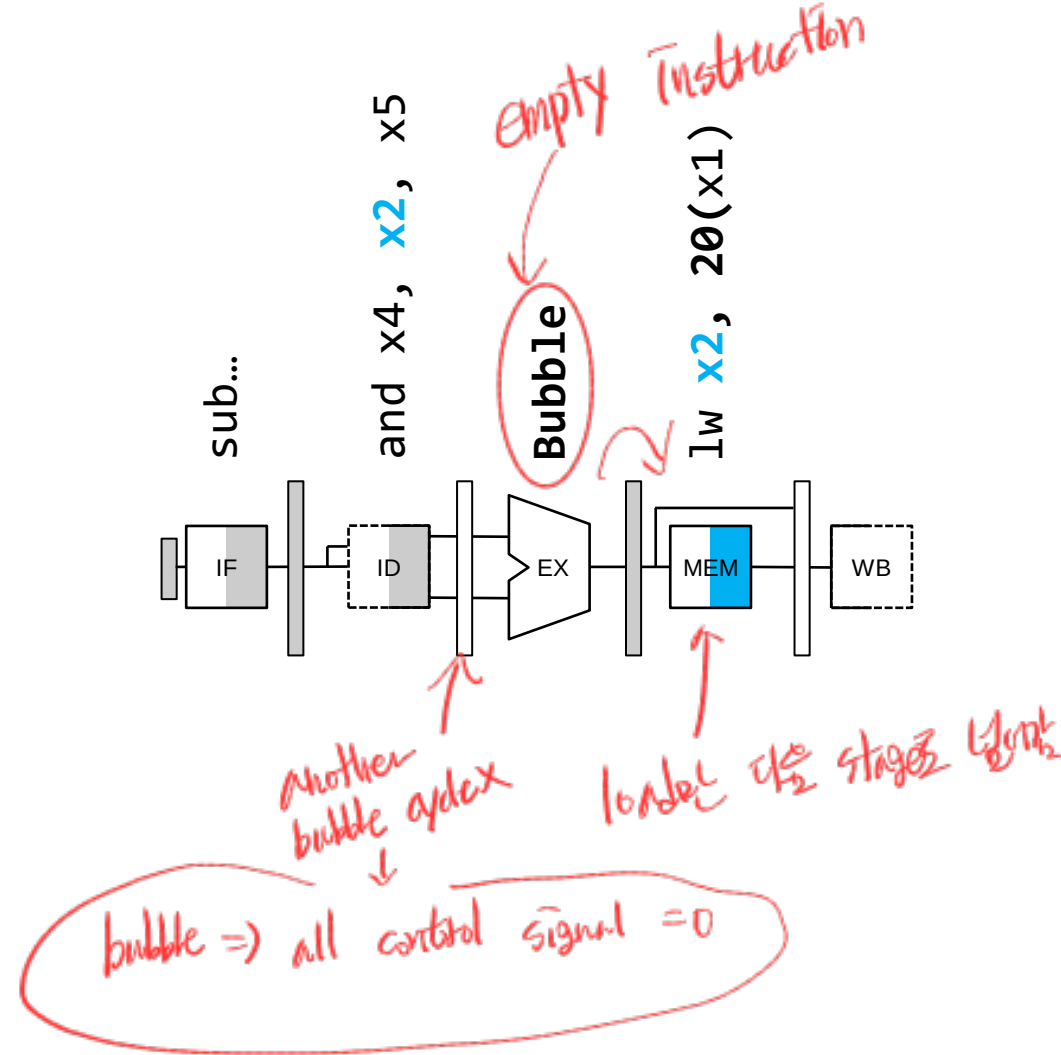
Stall for Load-Use Data Hazard – Different View: Cycle 2



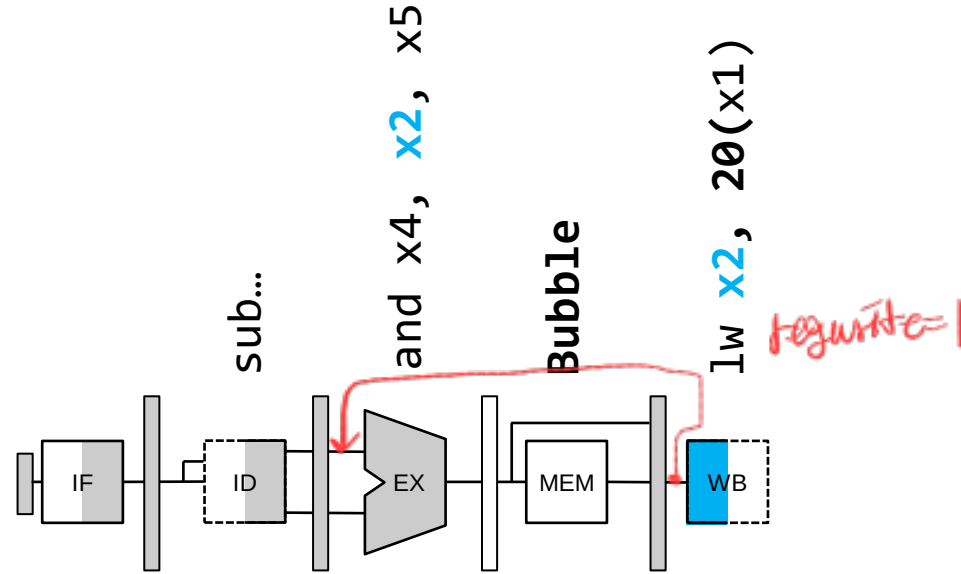
Stall for Load-Use Data Hazard – Different View: Cycle 3



Stall for Load-Use Data Hazard – Different View: Cycle 4



Stall for Load-Use Data Hazard – Different View: Cycle 5



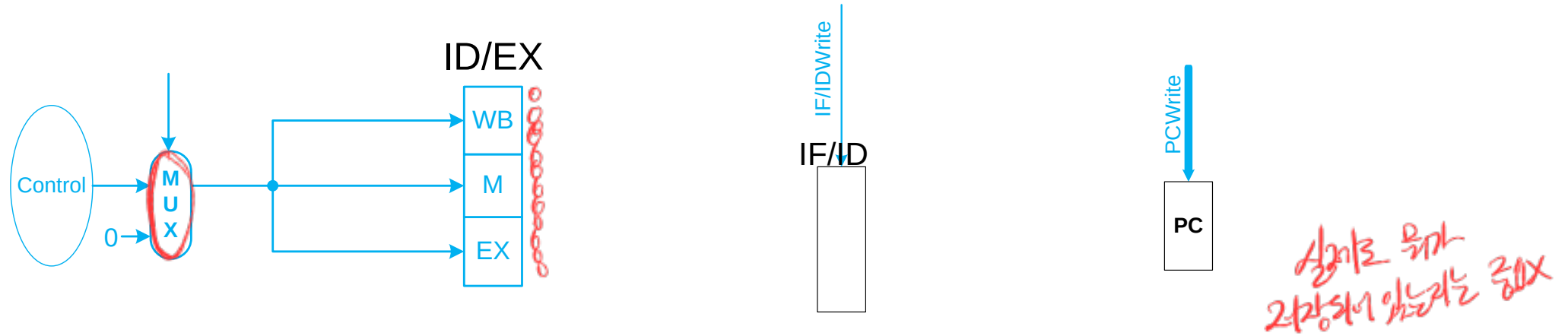
EX MEM
 ○○○○○○○○○
 ↑
 stage가 들어가기 전에
 stall이 들어간 후 실행

Bubble는
 stage를 따라 움직임

Adding a Bubble – What We Want

1. Pipeline Bubble in EX, MEM, WB stages: Skip
2. Repeat the ID stage 1 more cycle
3. Repeat the IF stage 1 more cycle

Adding a Bubble – How We Do It



1. EX, MEM, WB stages: Skip
 - All control signals (RegWrite, MemWrite, MemRead,...) : '0' *nothing happen datapath 이용*
2. Repeat the ID stage 1 more cycle
 - Do not change the IF/ID pipeline register → IF/IDWrite signal: '0' *normally = '1'*
3. Repeat the IF stage 1 more cycle
 - Do not change the PC → PCWrite signal: '0' *normally = '1'*

Load-Use Hazard Detection

아직 detect?

■ Check in ID stage

- ❖ ALU operand register numbers in ID stage are...
 - IF/ID.RegisterRs1, IF/ID.RegisterRs2
- ❖ Destination register for load instruction is
 - ID/EX.RegisterRd

아직 (22p430)와
상황이 다른, register
재사용 여부
22p441 simplified
condition은 역시 22p430X

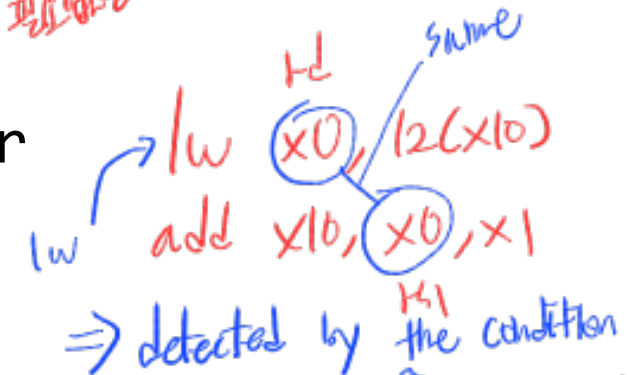
■ Load-use hazard when

- ❖ ID/EX.MemRead and

$((ID/EX.RegisterRd = IF/ID.RegisterRs1) \text{ or } (ID/EX.RegisterRd = IF/ID.RegisterRs2))$

only 'lw'만 사용 (bubbles는 2개만 들어
다른 instruction)
prevents multiple bubble cycle

not tightest bound
→ actual hardware는
bubbles의 발생을 방지 위해 3개

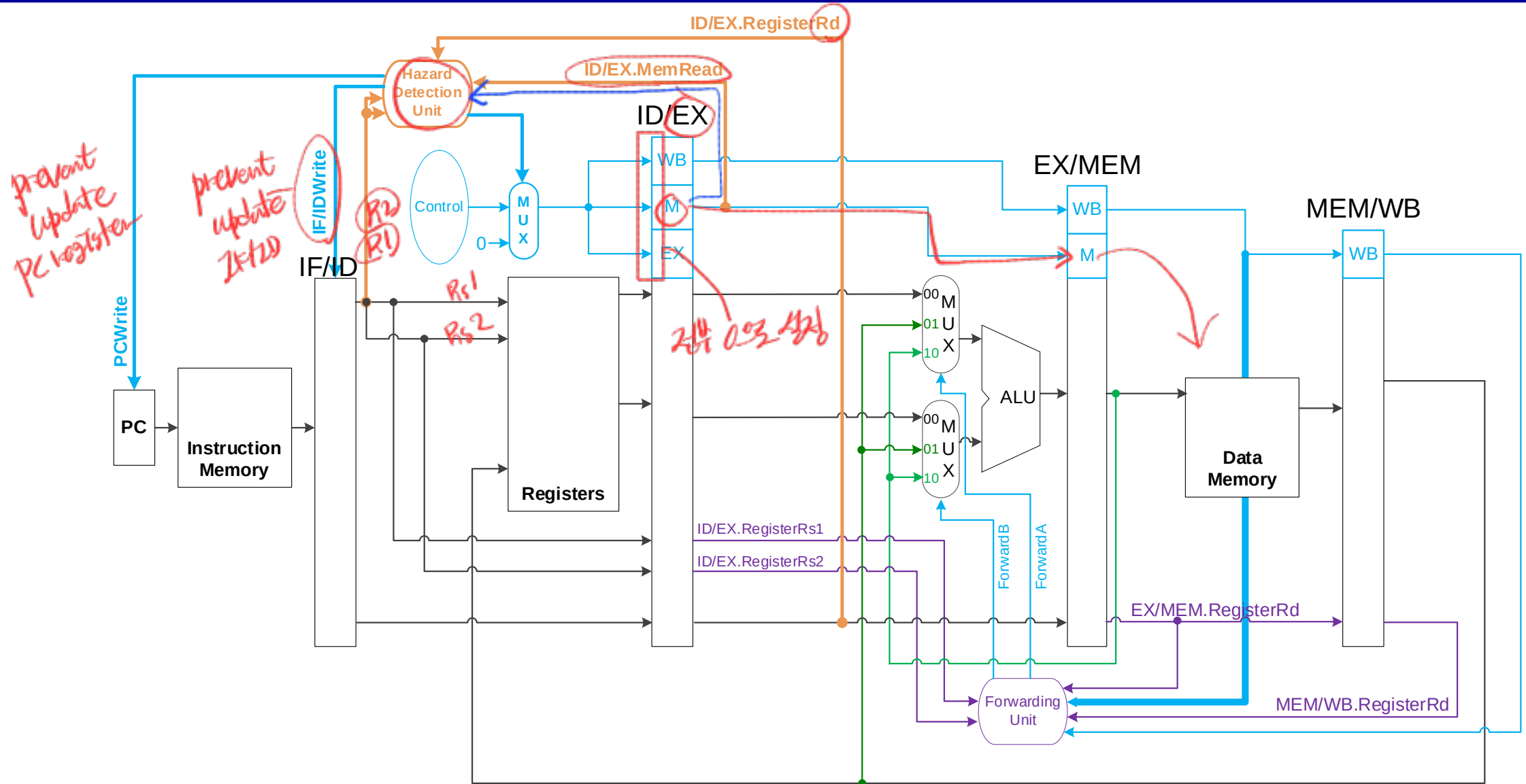


■ If detected, stall and insert bubble

1st Bubble Insertion
execution의 지연을 막기 위해
performance가 30% 이상으로
(아직 76%가 아닌 경우)
corner case인
2개 bubbles를 삽입

그러나 X0이 30
상대 bubble cycle
필요X → 30%를 증가하면?

Datapath with Hazard Detection



Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- $A = B + E; C = B + F;$

